
MANUALE TECNICO

SegnalaCity

Sistema digitale per la gestione delle segnalazioni urbane

Autore

Larosa Daniele

Data di consegna

12/06/2026

Github

https://github.com/gioLomb/segnalazioni_urbane.git

Indice

Sommario

Indice.....	2
1. Analisi.....	3
1.1 Analisi delle specifiche.....	3
1.1.1 Cittadini.....	3
1.1.2 Operatori Comunali.....	3
1.1.3 Amministratori.....	3
1.2 Architettura e Tecnologie.....	4
1.3 Avvio del Sistema e Autenticazione.....	5
1.4 Ciclo di Vita di una Segnalazione.....	6
1.5 Interfacce Dedicare per Ruolo.....	6
2. Requisiti Funzionali.....	7
3. Casi d'Uso.....	9
3.1 Login e accesso al sistema.....	9
3.2 Registrazione nuovo utente.....	10
3.3 Logout e invalidazione sessione.....	10
3.4 Inserimento nuova segnalazione.....	11
3.5 Visualizzazione segnalazioni (cittadino e operatore).....	12
3.6 Accettazione o rifiuto dell'assegnazione (operatore).....	12
3.7 Risoluzione di una segnalazione (operatore).....	13
3.8 Gestione segnalazioni e operatori (amministratore).....	13
3.9 Caricamento e consultazione dati geografici.....	14
3.10 Gestione trasversale: persistenza e sicurezza.....	15
4. Progettazione.....	16
4.1 Principali variabili, strutture dati e file.....	16
4.2 Librerie e Funzioni.....	32
4.2.1 report.h.....	33
4.2.2 user.h.....	36
4.2.3 session.h.....	37
4.2.4 geo.h.....	39
4.2.5 db.h.....	40
4.2.6 hash_table.h.....	42
4.2.7 rate_limiter.h.....	43
4.2.8 slab_allocator.h.....	43

4.2.9	client_manager.h.....	44
4.2.10	connection_handler.h.....	46
4.2.11	template.h.....	47
4.2.12	route_helpers.h.....	48
4.2.13	http_utils.h.....	50
4.2.14	server.h.....	52
5.	Codifica.....	52
5.1	Funzione di hashing — hash_key().....	53
5.2	Hash Table generica thread-safe.....	53
5.2.1	Inserimento — ht_set().....	53
5.2.2	Resize e rehashing — ht_resize().....	55
5.2.3	Lettura concorrente — ht_get().....	56
5.3	Slab Allocator.....	57
5.3.1	Creazione di un chunk — create_chunk().....	57
5.3.2	Allocazione — slab_pool_alloc().....	58
5.3.3	Deallocazione — slab_pool_free().....	59
5.4	Gestione connessioni — lista doppiamente concatenata.....	60
5.4.1	Inserimento in testa — client_manager_link().....	60
5.4.2	Rimozione — client_manager_unlink().....	61
5.4.3	Chiusura di tutte le connessioni — client_manager_close_all().....	61
5.5	Layer di rete — ciclo di lettura libuv.....	62
5.5.1	Allocazione lazy del buffer — on_buffer_alloc().....	62
5.5.2	Gestione della lettura e dispatch — on_read().....	63
5.6	Gestione delle sessioni.....	65
5.6.1	Creazione del token — session_create().....	65
5.6.2	Verifica con eviction lazy — session_verify().....	66
5.7	Rate limiter a finestra scorrevole.....	67
5.8	Cache JSON in-memory per i report.....	68
5.8.1	Lookup con eviction lazy — cache_find().....	68
5.8.2	Inserimento LRU-approx — cache_store().....	69
5.8.3	Lettura con integrazione cache — report_get_active_json().....	70
5.9	Ciclo di vita delle segnalazioni.....	71
5.9.1	Assegnazione admin — report_assign().....	72
5.9.2	Accettazione operatore — report_accept().....	72
5.9.3	Rifiuto operatore — report_reject().....	73
5.9.4	Feedback del cittadino — report_set_feedback().....	74

5.10 Autenticazione utente — user_authenticate()	75
5.11 Validazione geografica — geo_contains() e build_map_vars()	77
5.11.1 Test di appartenenza al bounding box — geo_contains()	77
5.11.2 Costruzione variabili mappa — build_map_vars()	78
5.12 Template engine — pre-parsing e rendering	78
5.12.1 Pre-parsing dei frammenti — parse_fragments()	78
5.12.2 Rendering — tpl_render()	80
6. Test	82
6.1 Test dei requisiti funzionali	82
6.2 Test di sicurezza	91
6.3 Test di performance	94
6.3.1 Metodologia	94
6.3.2 Script wrk utilizzato	94
6.3.3 Storico risultati	94
6.4 Test di copertura — lcov	96
Conclusioni	97

1. Analisi

Un'amministrazione comunale desidera sviluppare un sistema digitale denominato Segnalazioni Urbane, finalizzato a permettere ai cittadini di segnalare problemi presenti sul territorio — come buche stradali, impianti di illuminazione guasti, rifiuti abbandonati o altri disservizi urbani — con l'obiettivo di migliorare la gestione degli interventi e la qualità della vita in città.

Il sistema si propone come punto di incontro tra la cittadinanza e l'amministrazione locale: da un lato consente ai residenti di partecipare attivamente alla cura del territorio in modo semplice e immediato; dall'altro fornisce agli operatori comunali gli strumenti per organizzare, tracciare e risolvere i problemi segnalati in maniera efficiente e misurabile.

1.1 Analisi delle specifiche

Il sistema è progettato per rispondere alle esigenze di tre principali categorie di utenti: i cittadini, gli operatori comunali e gli amministratori. Ciascuna categoria dispone di un'interfaccia e di funzionalità dedicate, accessibili tramite autenticazione con credenziali personali.

1.1.1 Cittadini

I cittadini rappresentano gli utenti primari del sistema. Per accedere alle funzionalità è necessaria una registrazione con credenziali personali. Una volta autenticati, possono:

- inserire una nuova segnalazione specificando il tipo di problema (ad esempio buca stradale, illuminazione guasta, rifiuti abbandonati), la posizione geografica — validata tramite i dati geografici comunali — e una breve descrizione testuale;
- monitorare lo stato delle proprie segnalazioni nel tempo, che può essere In attesa, In lavorazione o Risolta;
- consultare le segnalazioni personalmente effettuate, con possibilità di filtraggio;
- feedback sull'efficacia degli interventi effettuati, contribuendo alla valutazione del servizio.

1.1.2 Operatori Comunali

Gli operatori comunali sono i responsabili della gestione e della risoluzione degli interventi. Il sistema mette a loro disposizione:

- la visualizzazione di tutte le segnalazioni ricevute, cioè assegnate dall'amministrazione comunale;
- la possibilità di filtrare ogni segnalazione;
- la possibilità di contrassegnare come risolto una segnalazione assegnata;
- la visualizzazione su mappa delle aree con maggiore concentrazione di problemi, così da prioritizzare gli interventi.

1.1.3 Amministratori

Gli amministratori dispongono di una visione globale del sistema. Possono accedere a tutti i dati del proprio comune, assegnare le segnalazioni agli operatori disponibili e analizzare lo storico degli interventi per pianificare azioni future.

La Tabella 1 riassume i ruoli del sistema e le relative funzionalità principali.

Ruolo	Funzionalità principali	Descrizione
Cittadino	Registrazione e accesso	Creazione account con credenziali

Ruolo	Funzionalità principali	Descrizione
		personali
	Inserimento segnalazione	Tipo di problema, posizione geografica, descrizione
	Monitoraggio stato	Visualizzazione stato: In attesa, In lavorazione, Risolta
	Consultazione e filtri	Ricerca per categoria o area geografica
	Feedback	Valutazione dell'efficacia degli interventi effettuati
Operatore Comunale	Gestione segnalazioni	Visualizzazione di tutte le segnalazioni ricevute
	Aggiornamento stato	Modifica progressiva dello stato degli interventi
	Monitoraggio e analisi	Coordinate delle segnalazioni su mappa
Amministratore	Storico e reportistica	Analisi dati storici per pianificazione interventi futuri
	Assegnazione segnalazione	Assegnazione segnalazioni ad operatori comunali

Tabella 1. Ruoli del sistema e funzionalità associate.

1.2 Architettura e Tecnologie

Il backend del sistema è implementato in linguaggio C, scelto per le sue prestazioni ottimali nella gestione delle connessioni di rete e per la modularità del codice. Il server adotta un'architettura event-driven a singolo thread basata sulla libreria libuv, che consente di gestire migliaia di connessioni simultanee tramite I/O non bloccante, senza l'overhead di un sistema multi-threaded. Il frontend è implementato in HTML e Javascript, e i dati dinamici vengono iniettati nelle pagine stesse prima di essere servite attraverso un sistema di template engine, alternativamente il server risponde alle richieste a endpoint API con dati in formato JSON.

Per la persistenza dei dati viene utilizzato SQLite3 configurato in modalità WAL (Write-Ahead Logging), che garantisce affidabilità transazionale e performance di lettura ottimali anche sotto carico concorrente. Il database memorizza le segnalazioni e gli utenti.

Il sistema integra dati geografici ISTAT in formato GeoJSON per validare le coordinate delle segnalazioni e per fornire una visualizzazione cartografica delle aree di intervento. Per la sicurezza, oltre alle credenziali utente classiche come username e password (registrata nel database con salt), vengono gestite anche le sessioni utente tramite token esadecimali casuali a 32 caratteri, memorizzati in cookie HTTP, senza trasmissione di credenziali nelle richieste successive al login. Un meccanismo di rate limiting per IP protegge il sistema da abusi e attacchi automatizzati.

La Tabella 2 descrive le principali tecnologie adottate e le motivazioni della scelta.

Componente	Tecnologia	Motivazione
Server applicativo	C con libuv (event loop asincrono)	Performance elevate, gestione concorrente delle connessioni
Database	SQLite3 (WAL mode)	Persistenza affidabile, semplicità di deployment, scalabile. Gestisce le entità users e reports
Autenticazione	Token esadecimale casuale (32 char) in cookie	Sessioni sicure senza trasmissione di credenziali in chiaro
Dati geografici	GeoJSON ISTAT + bounding box per comune	I dati geojson vengono indicizzati all'inizio (non appesantisce ogni richiesta) e utilizzati per validazione coordinate e rendering mappe.
Rate limiting	Sliding window per IP	Protezione da abusi e attacchi automatizzati
Template Engine	Frammentazione template HTML	Visualizzazione dinamica dati nelle pagine HTML
Parser HTTP	Libreria picohttpparser	Parsing HTTP/1.1 ad alte prestazioni senza copia dei buffer; nessuna dipendenza esterna aggiuntiva
Sistema di routing	Dispatch table statica	Separazione netta tra logica di instradamento e business logic
Indicizzazione in-	Hash table custom	Dati come sessioni attive,

Componente	Tecnologia	Motivazione
Pool di connessioni	Slab allocator a chunk fissi	Sotto carico elevato, ogni nuova connessione TCP richiede un'allocazione. Chiamare malloc per ogni connessione e free alla chiusura introduce frammentazione dell'heap e overhead. Lo slab allocator pre-alloca blocchi contigui (chunk) di 64 slot: le operazioni vengono effettuate in O(1)

Tabella 2. Componenti tecnologici del sistema.

1.3 Avvio del Sistema e Autenticazione

All'avvio del sistema, l'utente accede tramite un'interfaccia web che propone un modulo di login con campi per nome utente e password e la possibilità di creare un nuovo account. Premendo il pulsante di accesso, il sistema verifica le credenziali: in base al ruolo dell'utente autenticato, viene reindirizzato all'interfaccia dedicata (cittadino, operatore o amministratore).

La registrazione richiede l'inserimento delle seguenti informazioni:

- nome utente (username univoco nel sistema);
- password (gestita in modo sicuro lato server);
- comune di residenza o di competenza, validato tramite il database geografico.

Al termine dell'autenticazione, il server genera un token di sessione casuale a 32 caratteri esadecimali e lo invia al browser tramite cookie. Ogni richiesta successiva viene autenticata verificando la validità del token — che ha una durata massima di 24 ore — senza ulteriori accessi alle credenziali.

1.4 Ciclo di Vita di una Segnalazione

Una segnalazione attraversa un ciclo di vita definito, composto da tre stati progressivi. Ogni transizione di stato è tracciata con la marca temporale corrispondente, permettendo di misurare i tempi di risposta e di intervento in modo preciso.

La Tabella 3 illustra gli stati del ciclo di vita e il loro significato.

Stato	Codice interno	Significato
In attesa	STATUS_ACTIVE (0)	Segnalazione inviata, non ancora presa in carico
Assegnata	STATUS_ASSIGNED (1)	Segnalazione oggetto di richiesta di assegnazione da parte dell'amministrazione comunale ad un certo operatore
In lavorazione	STATUS_IN_PROGRESS (2)	Assegnazione segnalazione accettata dall'operatore che procederà all'intervento
Risolta	STATUS_RESOLVED (3)	Intervento completato dall'operatore

Tabella 3. Ciclo di vita di una segnalazione con stati e descrizioni.

Il sistema mantiene uno storico completo di tutte le segnalazioni e degli interventi effettuati. Questi dati aggregati permettono all'amministrazione di analizzare le criticità ricorrenti, identificare le aree geografiche con maggiore frequenza di problemi e pianificare interventi preventivi, migliorando nel tempo l'efficienza dei servizi comunali.

1.5 Interfacce Dedicate per Ruolo

Ciascun ruolo accede a un'interfaccia web dedicata, ottimizzata per le proprie esigenze operative.

Interfaccia Cittadino

L'interfaccia del cittadino permette di: inviare nuove segnalazioni con selezione del tipo di problema e della posizione su mappa, monitorare lo stato delle segnalazioni proprie, filtraggio, e fornire feedback sugli interventi completati.

Interfaccia Operatore Comunale

L'interfaccia dell'operatore mostra le segnalazioni attive assegnate allo specifico operatore, anche attraverso una mappa. Inoltre mostra le richieste di assegnazione da parte dell'amministrazione comunale che possono essere accettate o rifiutate. L'operatore può aggiornarne lo stato segnalandone la risoluzione.

Interfaccia Amministratore

L'interfaccia amministrativa offre una vista completa tramite mappa su tutte le segnalazioni del comune con la possibilità di assegnarle agli operatori comunali.

2. Requisiti Funzionali

I requisiti funzionali descrivono schematicamente le funzionalità implementate nel sistema Segnalazioni Urbane. A ciascun requisito è assegnato un codice univoco nella forma R00, utilizzato come riferimento nella sezione successiva per la descrizione dei casi d'uso.

Codice	Nome	Descrizione
R01	Visualizzazione pagina di login	Il sistema deve mostrare una pagina di accesso con campi per username e password e reindirizzare l'utente alla propria dashboard in base al ruolo, oppure mostrare un messaggio di errore in caso di credenziali errate.
R02	Registrazione nuovo utente	Il sistema deve permettere la creazione di un nuovo account inserendo username, password (minimo 6 caratteri), comune e ruolo. Il comune deve essere riconosciuto nel database geografico. È previsto un solo amministratore per comune.
R03	Autenticazione e gestione sessione	Il sistema deve autenticare l'utente tramite username e password, creare un token di sessione esadecimale casuale a 32 caratteri valido 24 ore e trasmetterlo tramite cookie.
R04	Logout e invalidazione sessione	Il sistema deve consentire all'utente autenticato di terminare la propria sessione.

Codice	Nome	Descrizione
R05	Visualizzazione dashboard contestuale	Il sistema deve mostrare una dashboard differente in base al ruolo dell'utente autenticato: pagina home per il cittadino, operatore o amministratore per i rispettivi ruoli.
R06	Inserimento nuova segnalazione	Il sistema deve permettere al cittadino autenticato di inserire una segnalazione specificando categoria, descrizione testuale e coordinate geografiche selezionate su mappa.
R07	Validazione geografica delle coordinate	Il sistema deve verificare che le coordinate inserite nella segnalazione appartengano al comune dell'utente tramite un test di bounding box sui dati GeoJSON ISTAT. Una segnalazione fuori dai limiti comunali viene rifiutata con messaggio di errore.
R08	Visualizzazione segnalazioni attive	Il sistema deve restituire le segnalazioni attive pertinenti all'utente: per il cittadino le proprie segnalazioni; per l'operatore le segnalazioni assegnate a lui con stato In lavorazione.
R09	Visualizzazione segnalazioni archiviate	Il sistema deve restituire le segnalazioni archiviate (stato Risolta): per il cittadino le proprie segnalazioni risolte; per l'operatore quelle da lui risolte.
R10	Visualizzazione tutte le segnalazioni (admin)	Il sistema deve permettere all'amministratore di visualizzare tutte le segnalazioni del proprio comune, indipendentemente dallo stato.
R11	Risoluzione di una segnalazione	Il sistema deve permettere all'operatore di segnare come Risolta una segnalazione precedentemente assegnata e accettata da sé stesso.
R12	Assegnazione segnalazione da parte dell'admin	Il sistema deve permettere all'amministratore di assegnare manualmente una segnalazione del proprio comune a un operatore appartenente alla stessa città. Vengono verificati l'esistenza del report, del comune e il ruolo dell'operatore.
R13	Visualizzazione lista operatori (admin)	Il sistema deve restituire all'amministratore l'elenco degli operatori registrati nel proprio comune in formato JSON, utilizzabile per la schermata di assegnazione.
R14	Lista comuni disponibili	Il sistema deve esporre un endpoint pubblico che restituisce la lista dei comuni disponibili per la

Codice	Nome	Descrizione
		registrazione, usando dati ufficiali ISTAT.
R15	Persistenza dei dati su database	Il sistema deve mantenere la persistenza di tutti i dati: utenti e segnalazioni su database SQLite, garantendo la durabilità tra riavvii del server. Sessioni, IP e coordinate città sono gestite in-memory.
R16	Protezione da rate limiting	Il sistema deve limitare il numero di richieste per indirizzo IP tramite un algoritmo a finestra scorrevole.
R17	Gestione degli errori HTTP	Il sistema deve restituire risposte HTTP appropriate per tutti i casi di errore: 400 per parametri mancanti o non validi, 401 per utente non autenticato, 403 per accesso non autorizzato, 404 per risorse non trovate, 405 per metodo non consentito, 409 per conflitti, 500 per errori interni.
R18	Caricamento dati geografici ISTAT	Il sistema deve caricare all'avvio il dataset GeoJSON dei comuni italiani, calcolare il bounding box e il centroide per ogni municipio e generare il file JSON della lista dei comuni, rendendo disponibili i dati per la validazione e il rendering delle mappe.
R19	Accettazione o rifiuto dell'assegnazione (operatore)	Il sistema deve permettere all'operatore di accettare o rifiutare una segnalazione assegnata dall'amministratore. In caso di accettazione la segnalazione passa allo stato In lavorazione; in caso di rifiuto torna allo stato In attesa e viene resa nuovamente disponibile per una nuova assegnazione.

Tabella 2. Requisiti funzionali del sistema Segnalazioni Urbane.

3. Casi d'Uso

I casi d'uso descrivono le interazioni tra gli utenti e il sistema per ciascun gruppo di requisiti. Per ogni caso d'uso sono riportate le pre-condizioni necessarie all'attivazione, le post-condizioni in caso di successo o fallimento, lo scenario di base e lo scenario alternativo.

3.1 Login e accesso al sistema

R01, R03, R05 Login e accesso al sistema	
Descrizione	Il sistema deve consentire all'utente registrato di autenticarsi tramite username e password e reindirizzarlo alla dashboard appropriata in base al proprio ruolo.

Pre-condizioni	• L'applicazione è avviata e il database è raggiungibile. • L'utente è già registrato nel sistema. • L'utente non è già autenticato (nessuna sessione attiva).
Post-condizioni	Successo: L'utente è autenticato; viene creato un token di sessione e impostato il cookie; l'utente viene reindirizzato alla propria dashboard (cittadino, operatore o amministratore). Fallimento: Il sistema mostra un messaggio di errore «Username o password errati» e rimane sulla pagina di login.
Scenario di base	L'utente accede alla pagina radice (/), inserisce username e password e clicca su Accedi. Il sistema verifica le credenziali, genera il token di sessione e reindirizza a /home con la dashboard contestuale al ruolo.
Scenario alternativo	Credenziali errate: il sistema mostra un messaggio di errore inline nella pagina di login senza reindirizzamenti. Utente già autenticato: la visita a / reindirizza direttamente a /home.

Tabella 3. Caso d'uso — Login e accesso al sistema (R01, R03, R05).

3.2 Registrazione nuovo utente

R02, R03 Registrazione nuovo utente	
Descrizione	Il sistema deve permettere la creazione di un nuovo account con username, password, comune e ruolo, verificando che i dati siano validi e univoci.
Pre-condizioni	• L'applicazione è avviata e il database è raggiungibile. • L'utente non dispone ancora di un account. • Il comune selezionato è presente nel database geografico ISTAT.
Post-condizioni	Successo: Il nuovo utente è salvato nel database con password cifrata; l'utente viene reindirizzato alla pagina di login. Fallimento: Il sistema mostra un messaggio di errore specifico: campo obbligatorio mancante, password troppo corta, comune non riconosciuto, username già in uso, o comune con amministratore già esistente.
Scenario di base	L'utente accede a /register, compila il modulo (username, password, comune dalla lista, ruolo) e invia. Il sistema valida i dati, verifica l'unicità dell'username e salva il record nel database.
Scenario alternativo	Username già in uso: messaggio «Username già in uso». Comune non presente nel geo-database: messaggio «Comune non riconosciuto». Tentativo di registrare un secondo amministratore per la stessa città: messaggio «Esiste già un'amministrazione comunale

per questa città».

Tabella 4. Caso d'uso — Registrazione nuovo utente (R02, R03).

3.3 Logout e invalidazione sessione

R04 Logout e invalidazione sessione	
Descrizione	Il sistema deve permettere all'utente autenticato di terminare la propria sessione in modo sicuro.
Pre-condizioni	- L'utente è autenticato e dispone di un token di sessione valido. - Il cookie di sessione è presente nel browser.
Post-condizioni	Successo: Il token viene rimosso dal server; il cookie viene cancellato (Max-Age=0); l'utente è reindirizzato alla pagina di login. Fallimento: Se il token non è presente nel cookie, il sistema esegue comunque il reindirizzamento alla pagina di login senza errori.
Scenario di base	L'utente clicca su Logout nella propria dashboard. Il server riceve la richiesta GET /logout, individua il token nel cookie, lo rimuove dalla tabella delle sessioni e invia una risposta con Set-Cookie che imposta Max-Age=0, eliminando il cookie dal browser.
Scenario alternativo	Token assente o già scaduto: il sistema reindirizza comunque a / senza produrre errori, comportamento idempotente.

Tabella 5. Caso d'uso — Logout e invalidazione sessione (R04).

3.4 Inserimento nuova segnalazione

R06, R07 Inserimento nuova segnalazione	
Descrizione	Il sistema deve permettere al cittadino autenticato di inviare una segnalazione di un problema urbano, con categoria, descrizione e posizione geografica verificata.
Pre-condizioni	- L'utente è autenticato con ruolo Cittadino. - Il comune dell'utente è presente nel database geografico. - L'utente si trova nella pagina /submit.
Post-condizioni	Successo: La segnalazione è salvata nel database con stato In attesa, timestamp di creazione e coordinate validate; l'utente viene reindirizzato alla home. Fallimento: Il sistema mostra un messaggio di errore: descrizione assente, coordinate fuori dal bounding box comunale, o errore interno del database.

Scenario di base	Il cittadino accede a /submit, seleziona la categoria (es. Buche stradali), inserisce la descrizione e clicca sulla mappa per selezionare le coordinate. Il sistema invia il form via POST /submit; il server valida il comune e il bounding box, inserisce il record e reindirizza a /home.
Scenario alternativo	Coordinate fuori dal comune: il sistema mostra «Le coordinate non appartengono alla tua città» e rimane sulla pagina /submit con i dati già inseriti. Categoria non inviata: il sistema assegna automaticamente la categoria «Altro». Operatore o admin che tenta l'accesso a /submit: viene reindirizzato a /home.

Tabella 6. Caso d'uso — Inserimento nuova segnalazione (R06, R07).

3.5 Visualizzazione segnalazioni (cittadino e operatore)

R08, R09 Visualizzazione segnalazioni (cittadino e operatore)	
Descrizione	Il sistema deve restituire le segnalazioni pertinenti all'utente: attive (non ancora risolte) e archiviate (risolte), con contenuto differenziato per ruolo.
Pre-condizioni	• L'utente è autenticato (cittadino o operatore). • Esistono segnalazioni associate all'utente nel database.
Post-condizioni	Successo: Viene restituito un array JSON con le segnalazioni pertinenti, incluse le marche temporali e le coordinate geografiche. Fallimento: Il sistema restituisce 401 Unauthorized se la sessione non è valida.
Scenario di base	La dashboard dell'utente effettua una chiamata GET /api/reports/active. Il server verifica la sessione, determina il ruolo e restituisce: <u>per</u> il cittadino, le sue segnalazioni con stato Attiva; per l'operatore, le segnalazioni da lui accettate con stato In lavorazione.
Scenario alternativo	Sessione scaduta o assente: risposta 401. Nessuna segnalazione corrispondente: array JSON vuoto []. Richiesta di segnalazioni archiviate: stessa logica su GET /api/reports/archived.

Tabella 7. Caso d'uso — Visualizzazione segnalazioni (cittadino e operatore) (R08, R09).

3.6 Accettazione o rifiuto dell'assegnazione (operatore)

R19 Accettazione o rifiuto dell'assegnazione (operatore)	
Descrizione	Il sistema deve permettere all'operatore di accettare o rifiutare una segnalazione assegnata dall'amministratore. La risposta dell'operatore determina la transizione di stato della segnalazione.

Pre-condizioni	• L'utente è autenticato con ruolo Operatore. • Esiste almeno una segnalazione assegnata all'operatore con stato In attesa di conferma. • L'assegnazione è stata effettuata in precedenza dall'amministratore (R12).
Post-condizioni	Successo: Accettazione: la segnalazione passa allo stato In lavorazione e viene associata all'operatore; la dashboard si aggiorna. Rifiuto: la segnalazione torna allo stato In attesa, il campo assegnazione viene azzerato e la segnalazione è nuovamente disponibile per una nuova assegnazione da parte dell'amministratore. Fallimento: Il sistema restituisce 403 se la segnalazione non è assegnata a questo operatore; 404 se la segnalazione non esiste; 409 se lo stato non è compatibile con l'operazione richiesta.
Scenario di base	L'operatore accede alla propria dashboard e visualizza le segnalazioni assegnate in attesa di conferma. Per ciascuna può cliccare «Accetta» o «Rifiuta». Il sistema invia la risposta al server, che aggiorna lo stato della segnalazione nel database e restituisce conferma.
Scenario alternativo	Operatore che tenta di rispondere a una segnalazione non assegnata a lui: risposta 403. Segnalazione già in lavorazione o risolta: risposta 409, stato non modificabile. Errore database: risposta 500.

Tabella 8. Caso d'uso — Accettazione o rifiuto dell'assegnazione (operatore) (R19).

3.7 Risoluzione di una segnalazione (operatore)

R11 Risoluzione di una segnalazione (operatore)	
Descrizione	Il sistema deve permettere all'operatore di segnare come risolta una segnalazione precedentemente assegnata e accettata da sé stesso, completando il ciclo di vita.
Pre-condizioni	• L'utente è autenticato con ruolo Operatore. • Esiste una segnalazione con stato In lavorazione assegnata e accettata da questo operatore.
Post-condizioni	Successo: Lo stato diventa Risolta; viene registrata la marca temporale di risoluzione; la segnalazione passa nell'archivio. Fallimento: Il sistema restituisce 403 se la segnalazione non è assegnata a questo operatore oppure è già risolta.
Scenario di base	L'operatore clicca su «Segna come risolta» sulla segnalazione in lavorazione. La dashboard invia la richiesta al server. Il server verifica che la segnalazione sia assegnata all'operatore corrente,

	aggiorna lo stato a Risolta con il timestamp di risoluzione e reindirizza alla dashboard.
Scenario alternativo	Segnalazione non assegnata a questo operatore: risposta 403. Segnalazione già risolta: risposta 403. Errore database: risposta 500.

Tabella 9. Caso d'uso — Risoluzione di una segnalazione (operatore) (R11).

3.8 Gestione segnalazioni e operatori (amministratore)

R10, R12, R13 Gestione segnalazioni e operatori (amministratore)	
Descrizione	Il sistema deve permettere all'amministratore di visualizzare tutte le segnalazioni del proprio comune, consultare l'elenco degli operatori e assegnare manualmente le segnalazioni.
Pre-condizioni	• L'utente è autenticato con ruolo Amministratore. • Esistono segnalazioni e/o operatori registrati nel comune dell'amministratore.
Post-condizioni	Successo: L'amministratore visualizza la mappa con tutte le segnalazioni del comune e può assegnare ciascuna a un operatore della stessa città; l'assegnazione viene confermata e la segnalazione rimane in attesa di conferma da parte dell'operatore. Fallimento: Il sistema restituisce 403 se l'utente non è amministratore; 404 se la segnalazione non esiste; 400 se l'operatore non è valido (ruolo errato o comune diverso); 409 se la segnalazione è già assegnata.
Scenario di base	L'amministratore accede alla dashboard, che carica le segnalazioni e gli operatori disponibili. Seleziona una segnalazione sulla mappa e sceglie un operatore dalla lista. Il server valida i dati, aggiorna il database e mette la segnalazione in attesa di conferma da parte dell'operatore assegnato.
Scenario alternativo	Operatore di un altro comune selezionato: risposta 400 «invalid operator». Segnalazione già assegnata: risposta 409. Report inesistente: risposta 404.

Tabella 10. Caso d'uso — Gestione segnalazioni e operatori (amministratore) (R10, R12, R13).

3.9 Caricamento e consultazione dati geografici

R14, R18 Caricamento e consultazione dati geografici	
Descrizione	Il sistema deve caricare all'avvio i dati GeoJSON ISTAT, costruire la tabella dei comuni con bounding box e centroide, e renderli disponibili per la registrazione e la validazione delle coordinate.

Pre-condizioni	• Il file data/comuni.json è presente e leggibile. • Il sistema è in fase di avvio.
Post-condizioni	Successo: La tabella interna dei comuni è popolata; il file data/cities.json è generato; i dati geografici sono disponibili per la validazione delle segnalazioni e il rendering delle mappe. Fallimento: Se il file GeoJSON non è leggibile o è malformato, il server termina con un errore fatale prima di accettare connessioni.
Scenario di base	All'avvio il server legge il file GeoJSON tramite memory-map, analizza ogni feature e calcola bounding box e centroide. Il file cities.json viene scritto. Successivamente l'endpoint pubblico serve la lista dei comuni per la pagina di registrazione.
Scenario alternativo	Comune non trovato durante la registrazione: messaggio «Comune non riconosciuto» e rifiuto della registrazione.

Tabella 11. Caso d'uso — Caricamento e consultazione dati geografici (R14, R18).

3.10 Gestione trasversale: persistenza e sicurezza

R15, R16, R17 Gestione trasversale: persistenza e sicurezza	
Descrizione	Il sistema deve garantire la persistenza dei dati, la protezione da abusi e la gestione corretta degli errori HTTP per tutte le operazioni.
Pre-condizioni	• Il database SQLite3 è accessibile. • Il server è avviato e il meccanismo di rate limiting è inizializzato.
Post-condizioni	Successo: I dati di utenti e segnalazioni sono persistiti sul database tra riavvii del server; ogni errore produce la risposta HTTP semanticamente corretta; gli IP che superano la soglia ricevono risposta 429. Fallimento: In caso di errore database, il sistema restituisce 500 con un messaggio esplicativo.
Scenario di base	Il server riceve una richiesta: verifica la soglia di rate limiting per l'IP mittente; se superata risponde 429. In caso contrario elabora la richiesta, interagisce con il database SQLite e restituisce la risposta appropriata. Sessioni, IP e coordinate città sono gestiti interamente in-memory.
Scenario alternativo	Metodo HTTP non supportato per un path esistente: risposta 405. Path inesistente: risposta 404. IP che supera la soglia di richieste: risposta 429 Too Many Requests.

Tabella 12. Caso d'uso — Gestione trasversale: persistenza e sicurezza (R15, R16, R17).

4. Progettazione

4.1 Principali variabili, strutture dati e file

La tabella seguente raccoglie tutte le strutture dati, le enumerazioni, le costanti e i file che compongono il sistema Segnalazioni Urbane, organizzati per categoria con separatori visivi. Per le strutture più importanti del dominio applicativo — User, Session, Report, CacheEntry, ClientCtx, Hash_Table, Template, Chunk — la descrizione è volutamente esaustiva per facilitare la comprensione dell'architettura.

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
Strutture dati — Dominio applicativo			
User	struct	Rappresentazione in memoria di un utente del sistema. È la struttura centrale del dominio: viene popolata al momento del login e usata anche come campo della struct Session. Il campo role determina quale interfaccia HTML e quali API sono accessibili.	userId: uint64_t username: char[32] passwordHash: char[33] salt: char[33] city: char[32] role: UserRole
Session	struct	Record di sessione memorizzato nella hash table in-memory. Incorpora una copia completa della struct User copiata al momento del login, così ogni route handler ottiene ruolo, username e città direttamente dalla sessione senza accedere al database. Il campo createdAt permette la verifica della scadenza (SESSION_MAX_AGE = 86 400 s) durante la validazione in session_verify(): le sessioni scadute vengono rimosse al primo accesso successivo, senza richiedere un thread di pulizia dedicato.	user: User createdAt: time_t
Report	struct	Rappresentazione in memoria di una singola segnalazione letta dal database. I campi sono ordinati per minimizzare il padding strutturale: prima i tipi a 64 bit (reportId, authorId, assignedTo), poi i double per le coordinate geografiche, poi i tre time_t del ciclo di vita (creazione, assegnazione, risoluzione), poi gli array char a lunghezza fissa e infine l'enum.	reportId: uint64_t authorId: uint64_t assignedTo: uint64_t lat: double lon: double createdAt: time_t assignedAt: time_t resolvedAt: time_t city: char[32] category: char[16] status: ReportStatus description: char[128]
CacheEntry	struct	Slot della cache JSON in-memory per le risposte delle query sui report. cachedAt = 0 identifica uno slot libero. Ogni voce è identificata dalla quadrupla (city, userId, isOperator, isArchived). Le voci	city: char[32] userId: uint64_t isOperator: bool isArchived: bool cachedAt: time_t

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		scadono dopo 5 secondi con eviction lazy al primo accesso successivo. I payload più grandi di CACHE_JSON_MAX (16 KB) vengono serviti direttamente dal database senza memorizzazione.	jsonLen: size_t json: char[16384]

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
CityGeo	struct	Dati geometrici calcolati per ogni comune durante il caricamento del dataset GeoJSON ISTAT in <code>geo_init()</code> . Il bounding box axis-aligned (latMin/latMax, lonMin/lonMax) è usato per la validazione delle coordinate inserite nella segnalazione: un punto fuori dal box viene rifiutato. Il centroide aritmetico è usato da per centrare automaticamente la mappa sul comune dell'utente nella pagina <code>/submit</code> .	latMin: double latMax: double lonMin: double lonMax: double centroidLat: double centroidLon: double
MapVars	struct	Stringhe decimali pre-formattate con i dati geografici di un comune, pronte per l'iniezione diretta nei template HTML/JavaScript delle mappe senza ulteriori conversioni.	lat: char[32] lon: char[32] bounds: char[128]
Strutture dati — Layer HTTP			
HttpRequest	struct	Rappresentazione di una richiesta HTTP in ingresso, prodotta dal parsing. I campi <code>method</code> , <code>path</code> e <code>body</code> sono slice non-NULL-terminate che puntano direttamente nel buffer di ricezione TCP: nessuna copia né heap allocation avviene durante il parsing. Ogni campo stringa è accompagnato dal corrispondente campo di lunghezza, obbligatorio per qualsiasi operazione su queste stringhe dato che non sono NUL-terminate.	method: const char* methodLen: size_t path: const char* pathLen: size_t headers: phr_header[32] numHeaders: size_t minorVersion: int body: const char* bodyLen: size_t
HttpResponse	struct	Rappresentazione di una risposta HTTP in uscita. Il campo <code>body</code> punta al buffer statico condiviso: questo è sicuro solo perché il server è single-threaded e processa una sola richiesta alla volta. I campi <code>setCookie</code> e <code>location</code> vengono inclusi come header aggiuntivi solo se non sono stringhe vuote, permettendo ai route handler di ometterli	statusCode: int contentType: const char* body: char* bodyLen: size_t setCookie: char[512] location: char[256]

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		semplicemente non scrivendoci nulla.	

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
WriteReq	struct	Struttura di supporto allocata sull'heap che raggruppa il descrittore di scrittura libuv (uv_write_t) e il payload HTTP completo — status line, header e body — in un unico blocco contiguo tramite flexible array. Il campo req deve essere il primo membro perché libuv esegue internamente il cast da WriteReq* a uv_write_t*. La memoria viene liberata interamente non appena il kernel ha consumato i dati, eliminando overhead di copia e double-free.	req: uv_write_t ctx: ClientCtx* keepAlive: int data: char[]
Strutture dati — Layer di rete e connessioni			
ClientCtx	struct	Stato completo di una singola connessione TCP attiva. Gestisce sia l'handle libuv per I/O (uv_tcp_t) sia il timer di keepalive (uv_timer_t). I bit-field closing e pendingCloses coordinano lo shutdown. La lista doppiamente concatenata (next/prev) è usata per il monitoraggio e lo shutdown ordinato di tutte le connessioni attive.	handle: uv_tcp_t timer: uv_timer_t closing: uint:1 pendingCloses: uint:2 next: ClientCtx* prev: ClientCtx* totalRead: size_t buffer: char*
SlabPool	struct	Handle pubblico dell'allocatore a blocchi fissi usato per i ClientCtx. Il campo chunksHead punta alla testa della lista doppiamente concatenata dei Chunk interni. blockSize è la dimensione visibile al chiamante; internalBsize include il back-pointer nascosto (sizeof(void*) + blockSize) che permette di trovare il Chunk proprietario (parent Chunk) in O(1) senza metadati aggiuntivi nella porzione utente del blocco.	blockSize: size_t internalBsize: size_t chunkCapacity: int chunksHead: void*
Chunk	struct	Nodo interno della lista del SlabPool. Ogni Chunk è un blocco contiguo di memoria suddiviso in slot di dimensione fissa; una free-list locale (localFree) permette	next: Chunk* prev: Chunk* localFree: void* /* free-list */ usedCount: int capacity: int

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		allocazione e free in $O(1)$ senza cercare slot liberi. Il campo <code>usedCount</code> traccia gli slot occupati (reference counting): quando scende a zero il Chunk viene liberato, tranne l'ultimo rimasto che viene trattenuto per evitare cicli ripetuti di <code>malloc/free</code> sotto carico basso. Il back-pointer nascosto all'inizio di ogni blocco permette di risalire al Chunk proprietario in $O(1)$.	<code>data: char[]</code>

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
Strutture dati — Hash table e rate limiter			
Hash_Table	struct	Tabella hash generica thread-safe con chaining separato (array di liste Entry). Protetta da un readers-writer lock POSIX (pthread_rwlock_t): più letture concorrenti sono consentite simultaneamente; le scritture acquisiscono il lock esclusivo. Usata in tre contesti: sessioni utente (chiave = token hex 32 char), rate limiter (chiave = stringa IP) e cache geografica (chiave = nome comune). Il seed per-tabella previene attacchi di hash-flooding. Supporta snapshot binari su file per opzionale persistenza tra riavvii del server.	pool: Entry** size: size_t capacity: size_t seed: unsigned long hashFunction: hash_func lock: pthread_rwlock_t
Entry	struct	Nodo di una catena nella Hash_Table. Chiave e valore sono buffer binari generici (void* + size_t), rendendo la tabella riutilizzabile per qualsiasi tipo di dato senza specializzazione. Il campo hash viene memorizzato per evitare il ricalcolo.	key: void* keySize: size_t value: void* size: size_t hash: unsigned long next: Entry*
RateEntry	struct	Stato per-IP del rate limiter a finestra scorrevole. Due contatori (finestra corrente e precedente) permettono di stimare il tasso di richieste senza conservare i timestamp individuali: il tasso stimato è $\text{countPrev} * (1 - \text{elapsed}) + \text{countCurr}$. La tabella viene azzerata automaticamente quando supera 10 000 entry, limitando l'uso di memoria in presenza di IP casuali.	countCurr: unsigned long countPrev: unsigned long windowStartTime: time_t
Strutture dati — Template engine e database			
Template	struct	Record del template engine. Il contenuto del file HTML viene caricato tramite mmap e copiato in un buffer heap (src). Al caricamento, il parser pre-calcola l'array fragments[] suddividendo il sorgente in segmenti	path: char* src: char* src_len: size_t fragments: Fragment* frag_count: int

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		FRAG_LITERAL (testo statico) e FRAG_VARIABLE (segnaposto {{KEY}} da sostituire). Al momento del rendering si iterano i frammenti in $O(n)$ nel numero di frammenti senza mai riesaminare il sorgente, rendendo il rendering efficiente anche per template grandi. I template vengono cercati per path.	

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
Fragment	struct	Unità elementare pre-calcolata dal parser dei template. Un frammento FRAG_LITERAL contiene un puntatore diretto nel buffer sorgente (ptr) e la sua lunghezza (len), così il rendering copia il testo con una sola memcpy senza allocazioni. Un frammento FRAG_VARIABLE contiene il nome della chiave (es. "USERNAME") che viene cercato linearmente nell'array TplVar[] passato a tpl_render().	type: FragType ptr: const char* len: size_t
FragType	enum	Discriminante del tipo di frammento di template. Determina il comportamento di del renderer per ogni unità pre-calcolata dal parser.	FRAG_LITERAL = 0 FRAG_VARIABLE = 1
TplVar	struct	Coppia chiave-valore per la sostituzione dei segnaposto {{KEY}} nei template HTML.	key: const char* value: const char*
DbCursor	struct	Handle opaco per l'iterazione dei risultati di una SELECT SQLite3. Incapsula il prepared statement e il flag hasRow per segnalare la disponibilità della riga corrente.	stmt: sqlite3_stmt* hasRow: bool
Route	struct	Voce della dispatch table del router HTTP. Mappa coppie (method, path) a funzioni handler.	method: const char* path: const char* handler: fn(HttpRequest*, HttpResponse*)
Altre enum			
UserRole	enum	Ruolo dell'utente nel sistema. Determina la dashboard mostrata al login (citizen_home, operator_map, admin_map), i permessi delle API e il sottoinsieme di dati visibili.	ROLE_CITIZEN = 0 ROLE_OPERATOR = 1 ROLE_ADMIN = 2
ReportStatus	enum	Stato del ciclo di vita di una segnalazione. La transizione è unidirezionale: ACTIVE→ASSIGNED→IN_PROGRESS→RESOLVED.	STATUS_ACTIVE = 0 STATUS_ASSIGNED = 1 STATUS_IN_PROGRESS = 2 STATUS_RESOLVED = 3
UserCol	enum	Indici delle colonne nel risultato	USER_COL_ID = 0

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		delle SELECT sulla tabella users.	USER_COL_USERNAME USER_COL_PASSWORD_HASH USER_COL_ROLE USER_COL_CITY USER_COL_SALT

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
ReportCol	enum	Indici delle colonne nel risultato delle SELECT sulla tabella reports.	REP_COL_ID = 0 REP_COL_AUTHOR_ID REP_COL_ASSIGNED_TO REP_COL_LAT / LON REP_COL_CITY REP_COL_CATEGORY REP_COL_DESCRIPTION REP_COL_STATUS REP_COL_CREATED/ ASSIGNED/ RESOLVED_AT
Costanti principali (config.h e moduli)			
PORT	costante	Porta TCP di ascolto.	Default: 8080
CLIENT_BUFFER_SIZE	costante	Dimensione del buffer di ricezione per connessione TCP. Limite pratico per la dimensione di una singola richiesta HTTP.	8 192 B (1<<13)
RESPONSE_BUFFER_SIZE	costante	Dimensione del buffer di risposta condiviso g_response_buffer.	262 144 B (256 KB)
MAX_CLIENTS	costante	Limite massimo di connessioni TCP simultanee.	16 384
KEEPALIVE_TIMEOUT	costante	Timeout inattività in secondi per connessioni TCP.	10 s
TOKEN_HEX_LENGTH	costante	Lunghezza del token di sessione.	32
SESSION_MAX_AGE	costante	Durata massima di una sessione in secondi.	86 400 s (24 h)
CACHE_TTL_SECONDS	costante	TTL delle voci in cache. Eviction lazy al primo accesso dopo la scadenza.	5 s
CACHE_MAX_ENTRIES	costante	Dimensione dell'array statico g_cache.	32 slot
CACHE_JSON_MAX	costante	Soglia oltre la quale una risposta JSON non viene memorizzata in	16 384 B (16 KB)

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		cache.	

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
RATE_LIMIT_RPS	costante	Soglia rate limiter (req/s per IP). Attivo solo se DEBUG_RATE_LIMIT=1.	100 req/s
MAX_TEMPLATES	costante	Dimensione dell'array dei template.	64
File sorgente C			
server.c / .h	file C	Entry point: inizializzazione db, geo, sessioni, rate limiter, template; binding TCP libuv; shutdown su SIGINT/SIGTERM.	main() hash_key() server_bind() server_shutdown()
route_handler.c / .h	file C	Dispatch table HTTP. Array routes[] di Route. Two-phase matching path→method.	routes[]: Route[] handle_request()
route_pages.c / .h	file C	Handler HTML: login, registrazione, home per ruolo, /submit, logout, CSS.	route_get_root/home/submit/logout() route_post_login/register/submit() route_static_css()
route_api.c / .h	file C	Handler JSON REST: report per tutti i ruoli, lista operatori, comuni. Verifica sessione e permessi di ruolo per ogni endpoint.	route_api_cities/operators() route_api_reports_active/archived/all() route_api_report_status() route_api_admin_assign()
user.c / .h	file C	CRUD utenti: registrazione, autenticazione, recupero per ID, JSON operatori, admin unici per città.	user_setup_table() user_register/authenticate() user_get_by_id() user_register_admin() user_get_operators_json()
report.c / .h	file C	CRUD segnalazioni: inserimento, assegnazione atomica, risoluzione con verifica proprietà. Cache JSON con TTL e invalidazione per città.	report_insert/assign/resolve() report_get_active/archived/all_city_json() report_cache_invalidate_city()
session.c / .h	file C	Sessioni in-memory su Hash_Table. Token hex casuale a 32 char.	session_init/destroy_all() session_create/verify/destroy()
geo.c / .h	file C	Modulo geografico: GeoJSON	geo_init/cleanup()

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		ISTAT, calcolo bounding box e centroide, Hash_Table, cities.json. popolamento generazione	geo_lookup/contains() build_map_vars()

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
db.c / .h	file C	Wrapper SQLite3 per operazioni DML e QL. Binding parametrizzato con format string.	db_init/close() db_cursor_open/next/text/int64/double/close() db_exec() db_last_insert_id()
hash_table.c / .h	file C	Hash table generica thread-safe con chaining e rwlock. Usata per sessioni, rate limiter, dati geografici.	ht_create/destroy() ht_set/get/delete() ht_foreach() ht_snapshot/load()
rate_limiter.c / .h	file C	Rate limiter a finestra scorrevole per IP. Auto-reset oltre 10 000 entry.	rate_limiter_init/destroy() rate_limit_check()
slab_allocator.c / .h	file C	Allocatore a blocchi fissi per ClientCtx. Chunk allineati 64 B, free-list locale O(1), chunk vuoti liberati tranne l'ultimo.	slab_pool_init/destroy() slab_pool_alloc/free()
client_manager.c / .h	file C	Registra connessioni TCP attive. SlabPool per allocazioni O(1). Lista doppiamente concatenata per monitoring e shutdown ordinato.	client_manager_init/destroy() client_manager_alloc/release() client_manager_link/unlink() client_manager_close_all()
connection_handler.c / .h	file C	Layer di rete libuv: gestisce le callback degli eventi primari come nuove connessioni, lettura richieste e scrittura della risposta	on_connection() on_buffer_alloc() on_read() on_write_complete() close_client() send_response()
template.c / .h	file C	Engine HTML minimale: mmap + pre-compilazione. In seguito nel rendering si itera sui frammenti di ogni template per iniettare dati dinamici.	tpl_load_files/unload_all() tpl_get/render()
route_helpers.c / .h	file C	Helper condivisi tra route_pages e route_api: verifica sessione, redirect, errori JSON/HTML, render template, errori per-pagina.	get_session_user() redirect() resp_json/html_error() resp_render_tpl() login/register/submit_error()
http_utils.c / .h	file C	Utility HTTP: confronto method/path, estrazione campi	http_is_request_method/path()

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
		URL-encoded, lettura cookie, escape HTML, formattazione header risposta.	get_field() http_request_cookie() html_escape() http_response_render()

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
File di dati e database			
segnalacity.db	DB SQLite3	Database principale dell'applicazione. Contiene le tabelle users e reports con indici.	Tabelle: users, reports
segnalacity.db-wal	DB SQLite3 WAL	Write-Ahead Log generato automaticamente da SQLite3 in WAL mode. Contiene le transazioni scritte ma non ancora integrate nel file principale. Elaborato al checkpoint.	Generato da SQLite3 Integrato al checkpoint
segnalacity.db-shm	DB SQLite3 SHM	File shared memory index generato automaticamente da SQLite3 in WAL mode. Coordina i lettori concorrenti durante scritture e checkpoint.	Generato da SQLite3 Coordinazione WAL readers
data/comuni.geojson	JSON (ISTAT)	Dataset GeoJSON dei comuni italiani (ISTAT). Caricato con mmap da geo_init(). Da ogni Feature vengono estratti nome comune e poligono per calcolare bounding box e centroide.	GeoJSON ISTAT comuni
data/cities.json	JSON (generato)	Lista JSON dei comuni disponibili, generata da geo_init() al primo avvio. Servita da GET /api/cities per popolare il menu della pagina di registrazione.	Array JSON di stringhe Es: ["Bari", "Roma", ...]
Template HTML e CSS (cartella templates/)			
login.html	template HTML	Pagina di accesso. Form username/password.	{{ERROR_BLOCK}} Route: GET /
register.html	template HTML	Pagina di registrazione. Campi username, password, comune (via /api/cities), ruolo. {{ERROR_BLOCK}} per errori di validazione.	{{ERROR_BLOCK}} Route: GET /register
citizen_home.html	template HTML	Dashboard cittadino. Mappa con segnalazioni attive e archiviate caricate via API. Pulsanti inserimento e logout.	{{USERNAME}}, {{CITY}} Route: GET /home (CITIZEN)
operator_map.	template	Dashboard operatore. Mappa con	{{USERNAME}}, {{CITY}}

Nome	Tipologia	Descrizione	Tipi / Campi / Valori
html	te HTML	segnalazioni assegnate. Presa in carico e risoluzione tramite POST /api/report/status.	Route: GET /home (OPERATOR)
admin_map.html	template HTML	Dashboard amministratore. Tutte le segnalazioni del comune. Assegnazione operatori tramite POST /api/admin/assign.	{{USERNAME}}, {{CITY}} Route: GET /home (ADMIN)
submit.html	template HTML	Form inserimento segnalazione. Mappa centrata sul comune tramite {{MAP_LAT}}, {{MAP_LON}}, {{MAP_BOUNDS}}.	{{USERNAME}}, {{CITY}} {{MAP_LAT}}, {{MAP_LON}} {{MAP_BOUNDS}}, {{ERROR_BLOCK}} Route: GET /submit
common.css	file CSS	Foglio di stile condiviso. Gestito come template per beneficiare di mmap e cache interna. Servito da GET /static/common.css.	Route: GET /static/common.css Content-Type: text/css

Tabella 4. Tipi di dato, strutture dati e file del sistema Segnalazioni Urbane.

4.2 Librerie e Funzioni

Di seguito sono descritte tutte le librerie progettate per il sistema Segnalazioni Urbane. Per ciascun file .h sono elencate le funzioni dichiarate con il relativo scopo, i parametri in ingresso e il tipo restituito. Le funzioni sono raggruppate per modulo di appartenenza.

4.2.1 report.h

report.h <i>Operazioni CRUD e serializzazione JSON per le segnalazioni urbane. Gestisce l'intero ciclo di vita delle segnalazioni, la cache JSON in-memory e il feedback del cittadino.</i>			
Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
report_setup_table()	Crea la tabella reports e i suoi indici nel database SQLite se non esistono.	– (nessun parametro)	int – 0 successo, -1 errore SQL
report_insert()	Inserisce una nuova segnalazione con	authorId: uint64_t – ID	uint64_t – ID generato,

	stato ACTIVE e restituisce l'ID generato.	cittadino lat, lon: double – coordinate city: const char* – comune category: const char* – categoria desc: const char* – descrizione	0 su errore
report_assign()	Assegna una segnalazione ACTIVE a un operatore (azione admin). Imposta stato ASSIGNED. Guardia atomica: fallisce se già assegnata o non ACTIVE.	reportId: uint64_t – ID segnalazione operatorId : uint64_t – ID operatore	int – 1 assegnata, 0 guardia non soddisfatta, -1 errore
report_accept()	L'operatore accetta l'assegnazione: ASSIGNED → IN_PROGRESS. Verifica che la segnalazione sia ASSIGNED e assegnata all'operatore chiamante.	reportId: uint64_t – ID segnalazione operatorId : uint64_t – ID operatore	int – 1 accettata, 0 guardia non soddisfatta, -1 errore
report_reject()	L'operatore rifiuta l'assegnazione: ASSIGNED → ACTIVE, campo assigned_to azzerato. Il rifiuto permette una nuova assegnazione da parte dell'admin.	reportId: uint64_t – ID segnalazione operatorId : uint64_t – ID operatore	int – 1 rifiutata, 0 guardia non soddisfatta, -1 errore

<code>report_resolve()</code>	Marca una segnalazione IN_PROGRESS come RESOLVED. Verifica che sia assegnata all'operatore chiamante per impedire risoluzioni incrociate.	reportId: uint64_t – ID segnalazione operatorId: uint64_t – ID operatore	int – 1 risolta, 0 guardia non soddisfatta, -1 errore
<code>report_set_feedback()</code>	Registra il feedback (1–5 stelle) del cittadino autore su una segnalazione RESOLVED. Può essere impostato una sola volta.	reportId: uint64_t – ID segnalazione authorId: uint64_t – ID cittadino autore stars: int – valutazione 1–5	int – 1 salvato, 0 guardia non soddisfatta, -1 errore
<code>report_get_active_json()</code>	Serializza in JSON le segnalazioni attive pertinenti all'utente. Operatore: IN_PROGRESS assegnate a userId. Cittadino: proprie con stato < RESOLVED. Servite dalla cache se disponibili.	buf: char* – buffer output max: size_t – capacità buffer userId: uint64_t – ID utente city: const char* – chiave cache isOperator: bool – tipo utente	size_t – byte scritti in buf, 0 su errore
<code>report_get_archived_json()</code>	Serializza in JSON le segnalazioni risolte pertinenti all'utente. Stessa logica di	buf: char* – buffer output max: size_t –	size_t – byte scritti in buf, 0 su

	get_active_json ma filtra su STATUS_RESOLVED.	capacità buffer userId: uint64_t – ID utente city: const char* – chiave cache isOperator : bool – tipo utente	errore
report_get_all_city_json()	Serializza in JSON tutte le segnalazioni di una città (vista admin). Non cached; restituisce ogni segnalazione indipendentemente dallo stato.	buf: char* – buffer output max: size_t – capacità buffer city: const char* – nome comune	size_t – byte scritti in buf, 0 su errore
report_get_by_id()	Recupera una singola segnalazione per ID dal database e popola la struct Report.	reportId: uint64_t – ID da cercare out: Report* – struct da popolare	bool – true se trovata, false altrimenti
report_count_active()	Restituisce il numero totale di segnalazioni non ancora risolte nel database.	– (nessun parametro)	int – conteggio segnalazioni con status < RESOLVED
report_to_json()	Serializza una singola struct Report in stringa JSON nel buffer dest.	r: const Report* – segnalazione sorgente dest: char* –	bool – true se il JSON è rientrato in dest, false su

		buffer output destSize: size_t – capacità buffer	overflow
<code>report_cache_invalidate_city()</code>	Rimuove dalla cache in-memory tutte le voci relative a una città, chiamata dopo ogni scrittura (insert, assign, resolve).	city: const char* – città da invalidare authorId: uint64_t – ID autore per voci citizen- scoped	void

Tabella 5. Funzioni dichiarate in `report.h`.

4.2.2 user.h

user.h

Gestione degli utenti: registrazione con salt casuale e hashing salted, autenticazione con retrocompatibilità legacy, recupero per ID, predicati di ruolo e serializzazione JSON degli operatori.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>user_setup_table()</code>	Crea la tabella users e aggiunge la colonna salt tramite ALTER TABLE se mancante (migrazione per database esistenti).	– (nessun parametro)	int – 0 successo, -1 errore
<code>user_register()</code>	Registra un nuovo utente generando un salt casuale, calcolando hash(salt+password) e salvando entrambi nel database.	username: const char* plainPassword: const char* city: const char* role: UserRole	int – 0 successo, -1 errore (es. username duplicato)

<code>user_register_admin()</code>	Registra un amministratore imponendo il vincolo di unicità per città: fallisce con -2 se esiste già un admin per quella città.	username: const char* plainPassword: const char* city: const char*	int - 0 successo, -1 errore generico, -2 admin già esistente per città
<code>user_authenticate()</code>	Autentica un utente. Supporta sia hash salted (nuovi) sia hash non-salted (legacy). Popola *out con i dati utente in caso di successo.	username: const char* plainPassword: const char* out: User* - struct da popolare	bool - true se credenziali valide, false altrimenti
<code>user_get_by_id()</code>	Recupera un utente dal database tramite chiave primaria.	id: uint64_t - ID utente out: User* - struct da popolare	bool - true se trovato, false altrimenti
<code>user_is_operator()</code>	verifica se l'utente ha ruolo ROLE_OPERATOR.	u: const User* - utente da verificare	bool - true se operatore
<code>user_is_admin()</code>	verifica se l'utente ha ruolo ROLE_ADMIN.	u: const User* - utente da verificare	bool - true se amministratore
<code>user_get_operators_json()</code>	Serializza in JSON l'elenco degli operatori registrati per una città. Usato dall'endpoint GET /api/operators per popolare la lista di assegnazione dell'admin.	buf: char* - buffer output max: size_t - capacità buffer city: const char* - città da filtrare	size_t - byte scritti in buf

Tabella 6. Funzioni dichiarate in user.h.

4.2.3 session.h

session.h

Gestione delle sessioni in-memory tramite hash table. Le sessioni sono identificate da un token hex casuale a 32 caratteri memorizzato in un cookie HTTP. Ogni sessione incorpora la struct User completa per evitare round-trip al database.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>session_init()</code>	Alloca la hash table interna delle sessioni. Deve essere chiamata una sola volta all'avvio, prima di qualsiasi altra funzione session_*.	– (nessun parametro)	<code>int</code> – 0 successo, -1 su errore di allocazione
<code>session_destroy_all()</code>	Dealloca la hash table e invalida tutte le sessioni attive. Chiamata allo shutdown del server.	– (nessun parametro)	<code>void</code>
<code>session_create()</code>	Crea una nuova sessione per l'utente autenticato generando un token casuale. In caso di collisione (rarissima) riprova automaticamente.	user: const User* – utente da associare (copiato) outToken: char* – buffer	<code>bool</code> – true su successo, false su errore di allocazione
<code>session_verify()</code>	Verifica un token di sessione. Eviction lazy: le sessioni scadute vengono rimosse al primo accesso.	token: const char* – token hex dalla Cookie header outUser: User* – destinazione dati utente (può essere NULL)	<code>bool</code> – true se sessione valida e non scaduta
<code>session_destroy()</code>	Rimuove una singola sessione dalla tabella (logout esplicito).	token: const char* – token hex della sessione da rimuovere	<code>void</code>

Tabella 7. Funzioni dichiarate in session.h.

4.2.4 geo.h

geo.h

Modulo geografico per l'indicizzazione e la validazione delle coordinate. Gestisce una hash table interna indicizzata per nome comune; carica i dati dal GeoJSON ISTAT all'avvio tramite memory-map.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
geo_init()	Carica la geometria dei comuni dal file GeoJSON ISTAT (memory-map), calcola bounding box e centroide per ogni municipio e popola la hash table interna. Scrive opzionalmente l'array JSON dei nomi comuni in citiesOut.	geojsonPath: const char* – percorso file GeoJSON ISTAT citiesOut: const char* – percorso output cities.json (NULL = ometti)	int – numero di comuni caricati, -1 su errore fatale
geo_cleanup()	Rilascia tutte le risorse allocate da geo_init(). Sicuro da chiamare anche se geo_init() non è mai stata invocata.	– (nessun parametro)	void
geo_lookup()	Cerca i dati geometrici di un comune per nome nella tabella interna e popola la struct CityGeo.	city: const char* – nome del comune out: CityGeo* – struct da popolare	bool – true se trovato, false altrimenti
geo_contains()	Test di appartenenza al bounding box :verifica se una coppia lat/lon è all'interno dei limiti del comune. Approssimazione: il bounding box è più grande del poligono reale.	geo: const CityGeo* – geometria del comune lat: double – latitudine del punto lon: double – longitudine del punto	bool – true se il punto è nel bounding box
build_map_vars()	Popola la struct MapVars con stringhe decimali pre-formattate (centroide e bounds)	cityName: const char* – nome del comune mv: MapVars* – struct	void

	pronte per l'iniezione nei template HTML.	da popolare	
--	---	-------------	--

Tabella 8. Funzioni dichiarate in geo.h.

4.2.5 db.h

db.h

Wrapper ad alto livello per SQLite3. Fornisce un'API a cursore per le SELECT e db_exec() per DML/DDL. Il binding dei parametri usa un format string compatto.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
db_init()	Apri o crea il database SQLite al percorso indicato.	path: const char* – percorso del file .db	int – 0 successo, -1 se il database non può essere aperto
db_close()	Chiude la connessione globale al database.	– (nessun parametro)	void
db_errmsg()	Restituisce il messaggio di errore SQLite più recente.	– (nessun parametro)	const char* – stringa di errore, "no db" se non inizializzato
db_last_insert_id()	Restituisce il rowid generato dall'ultimo INSERT.	– (nessun parametro)	int64_t – rowid, 0 se nessuna connessione aperta
db_changes()	Restituisce il numero di righe modificate dall'ultimo statement DML.	– (nessun parametro)	int – numero di righe
db_cursor_open()	Prepara ed esegue una SELECT con binding parametrizzato e restituisce un	sql: const char* – query SELECT fmt: const char* – format string parametri	DbCursor* – cursore su successo, NULL su errore

	cursore iterabile. fmt descrive i parametri: 's'=string, 'i'=int, 'l'=int64, 'f'=double, 'n'=NULL.	(NULL se nessuno): argomenti variabili corrispondenti a fmt	
db_cursor_next()	Avanza il cursore alla riga successiva del result set.	cursor: DbCursor* – cursore attivo	bool – true se una riga è disponibile, false a EOF o su errore
db_cursor_text()	Legge il valore testuale della colonna indicata dalla riga corrente del cursore.	cursor: DbCursor* col: int – indice colonna (base 0)	const char* – valore stringa, "" se NULL o cursore non valido
db_cursor_int64()	Legge il valore intero a 64 bit della colonna indicata.	cursor: DbCursor* col: int – indice colonna	int64_t – valore, 0 se NULL o cursore non valido
db_cursor_double()	Legge il valore double della colonna indicata.	cursor: DbCursor* col: int – indice colonna	double – valore, 0.0 se NULL o cursore non valido
db_cursor_close()	Finalizza lo statement e libera il cursore. Il puntatore diventa non valido dopo questa chiamata.	cursor: DbCursor* – cursore da chiudere	void
db_exec()	Prepara ed esegue un singolo statement DML/DDI con binding parametrizzato (stesso formato di db_cursor_open).	sql: const char* – statement SQL fmt: const char* – format string parametri: argomenti variabili	int – 0 successo, -1 errore

Tabella 9. Funzioni dichiarate in db.h.

4.2.6 hash_table.h

hash_table.h

Hash table generica thread-safe con chaining separato e readers-writer lock POSIX. Usata per sessioni utente, rate limiter e cache geografica. Supporta chiavi e valori binari arbitrari.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
ht_create()	Alloca e inizializza una nuova hash table con la capacità e la funzione di hashing indicate.	initialCapacity: size_t – numero iniziale di bucket hashFunction: hash_func – puntatore alla funzione di hash	Hash_Table* – nuova tabella, NULL su errore
ht_set()	Inserisce o aggiorna una coppia chiave-valore. Acquisisce il write lock. Incrementa il contatore se la chiave è nuova.	table: Hash_Table* key: void* – buffer chiave keySize: size_t value: void* – buffer valore valueSize: size_t	int – 1 successo, 0 errore
ht_get()	Recupera una copia del valore associato alla chiave nel buffer di destinazione. Acquisisce il read lock.	table: Hash_Table* key: void*, keySize: size_t destBuffer: void* – output destSize: size_t – capacità output	int – 1 trovato, 0 non trovato
ht_delete()	Rimuove una voce dalla tabella liberando chiave e valore. Acquisisce il write lock.	table: Hash_Table* key: void*, keySize: size_t	int – 1 trovato e rimosso, 0 non trovato
ht_destroy()	Distrugge la tabella liberando tutte le entry, il pool e la struct. Salva opzionalmente uno snapshot prima della distruzione.	table: Hash_Table* persistenceFilePath: const char* – percorso snapshot (NULL = ometti)	void
ht_foreach()	Itera su tutte le voci della tabella applicando il callback cb a ciascuna. Sicuro per unlink/close del nodo corrente.	table: Hash_Table* cb: ht_foreach_cb – funzione callback userdata: void* – puntatore passato al callback	void

<code>ht_snapshot()</code>	Serializza tutte le voci correnti in un file binario per la persistenza opzionale tra riavvii.	table: Hash_Table* path: const char* – percorso file output	void
----------------------------	--	--	------

Tabella 10. Funzioni dichiarate in `hash_table.h`.

4.2.7 rate_limiter.h

rate_limiter.h <i>Rate limiter a finestra scorrevole per indirizzo IP. Traccia due contatori (finestra corrente e precedente) per stimare il tasso senza conservare timestamp individuali.</i>			
Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>rate_limiter_init()</code>	Alloca la hash table interna per il tracciamento per-IP. Deve essere chiamata una volta all'avvio.	– (nessun parametro)	int – 0 successo, -1 su errore di allocazione
<code>rate_limiter_destroy()</code>	Libera la hash table interna.	– (nessun parametro)	void
<code>rate_limit_check()</code>	Verifica se la richiesta per l'IP indicato è entro il limite. La tabella viene azzerata automaticamente oltre 10 000 entry. Restituisce sempre 1 se <code>DEBUG_RATE_LIMIT=0</code>	ip: const char* – stringa IPv4 NUL-terminated	int – 1 se consentita, 0 se rifiutata

Tabella 11. Funzioni dichiarate in `rate_limiter.h`.

4.2.8 slab_allocator.h

slab_allocator.h <i>Allocatore a blocchi fissi (slab allocator) per oggetti della stessa dimensione. Riduce la frammentazione heap e garantisce alloc/free O(1) allocando memoria in chunk e suddividendola in slot.</i>			
Funzione	Scopo	Parametri in ingresso	Tipo di ritorno

<code>slab_pool_init()</code>	Inizializza il pool per blocchi di blockSize byte. Alloca il primo chunk immediatamente così il pool è pronto all'uso al ritorno.	pool: SlabPool* – pool da inizializzare blockSize: size_t – dimensione di ogni blocco chunkCapacity: int – slot per chunk	int – 0 successo, -1 se l'allocazione iniziale fallisce
<code>slab_pool_destroy()</code>	Rilascia tutta la memoria posseduta dal pool..	pool: SlabPool* – pool da distruggere	void
<code>slab_pool_alloc()</code>	Alloca un blocco dal pool. La memoria NON è inizializzata a 0. Alloca un nuovo chunk internamente se tutti gli esistenti sono pieni.	pool: SlabPool* – pool sorgente	void* – puntatore al blocco, NULL se esaurita la memoria
<code>slab_pool_free()</code>	Restituisce un blocco al pool per il riuso. I chunk che diventano completamente vuoti vengono liberati, tranne l'ultimo (trattenuto per evitare thrashing).	pool: SlabPool* – pool di appartenenza ptr: void* – blocco da restituire	void

Tabella 12. Funzioni dichiarate in slab_allocator.h.

4.2.9 client_manager.h

`client_manager.h`

Registry e gestore della memoria per le connessioni TCP attive. Combina uno slab allocator per alloc/release O(1) e una lista doppiamente concatenata globale per il monitoring e lo shutdown ordinato.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>client_manager_init()</code>	Inizializza lo	– (nessun	int – 0

	slab pool interno e azzerà lo stato globale.	parametro)	successo, -1 su errore di allocazione
<code>client_manager_destroy()</code>	Rilascia tutta la memoria slab e azzerà il gestore. Le connessioni devono essere chiuse prima della chiamata.	– (nessun parametro)	void
<code>client_manager_alloc()</code>	Alloca un ClientCtx azzerato dallo slab pool.	– (nessun parametro)	ClientCtx* – nuovo contesto, NULL se memoria esaurita
<code>client_manager_release()</code>	Restituisce un ClientCtx allo slab pool e libera il suo buffer di ricezione. Richiede pendingCloses == 0.	ctx: ClientCtx* – contesto da rilasciare	void
<code>client_manager_link()</code>	Inserisce una connessione in testa alla lista attiva globale.	ctx: ClientCtx* – contesto da collegare	void
<code>client_manager_unlink()</code>	Rimuove una connessione dalla lista attiva globale.	ctx: ClientCtx* – contesto da scollegare	void
<code>client_manager_active_cou</code>	Restituisce il	– (nessun	int –

<code>nt()</code>	numero di connessioni attualmente attive.	parametro)	numero di ClientCtx collegati
<code>client_manager_close_all()</code>	Itera su tutte le connessioni attive applicando fn a ciascuna.	fn: void(*) (ClientCtx*) – callback di chiusura	void

Tabella 13. Funzioni dichiarate in `client_manager.h`.

4.2.10 connection_handler.h

connection_handler.h Layer di rete libuv: accettazione connessioni, allocazione buffer, accumulo bytes, parsing HTTP, rate limiting, dispatch e invio risposta. Espone i quattro callback libuv e il helper di chiusura.			
Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>on_connection()</code>	Callback accettazione libuv (passato a <code>uv_listen()</code>). Alloca ClientCtx, inizializza gli handle TCP e timer, accetta il socket, arma il timer keepalive e avvia la lettura.	server: uv_stream_t* – handle server in ascolto status: int – 0 successo, negativo su errore libuv	void
<code>close_client()</code>	Avvia la chiusura di un client (TCP + timer). Protetto da <code>ctx->closing</code> per evitare double-close. La memoria viene rilasciata nell' <code>on_close</code> quando entrambi i callback libuv sono stati eseguiti.	ctx: ClientCtx* – contesto da chiudere	void
<code>on_buffer_alloc()</code>	Callback allocazione buffer libuv. Alloca il buffer del ClientCtx alla prima chiamata; nelle successive restituisce la coda inutilizzata. Imposta buf-	handle: uv_handle_t* – handle TCP (data = ClientCtx*) suggested: size_t – dimensione suggerita	void

	>base=NULL quando il buffer è pieno.	(ignorata) buf: uv_buf_t* – descrittore output da popolare	
<code>on_read()</code>	Callback lettura libuv. Accumula bytes in buffer del client finché la richiesta non è completa, poi effettua il parsing HTTP, verifica il rate limit, esegue il dispatch con <code>handle_request()</code> e invia la risposta. Chiude la connessione su errore di rete, parsing o rate limit.	stream: uv_stream_t* – stream TCP (data = ClientCtx*) nread: ssize_t – byte letti; negativo = EOF/errore buf: const uv_buf_t* – descrittore buffer (solo informativo)	<code>void</code>

Tabella 14. Funzioni dichiarate in `connection_handler.h`.

4.2.11 template.h

`template.h`

Template engine HTML minimale. Carica i file tramite memory-map, li pre-compila in un array di frammenti LITERAL/VARIABLE e li serve dalla cache interna. `tpl_render()` sostituisce i segnaposto `{{KEY}}` in $O(n)$ sui frammenti.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>tpl_load_files()</code>	Carica più template da file (lista di percorsi terminata da NULL). Usa l'attributo sentinel per il controllo a compile-time. Restituisce 0 se tutti i file sono stati caricati.	first_path: const char* – primo percorso ...: altri percorsi, terminati da NULL (sentinel)	<code>int</code> – 0 successo, -1 se almeno un file non è stato caricato
<code>tpl_unload_all()</code>	Libera tutti i template e svuota la cache interna.	– (nessun parametro)	<code>void</code>
<code>tpl_get()</code>	Cerca un	path: const char* –	<code>const</code>

	template nella cache per percorso (la stessa stringa usata al caricamento).	percorso del template (es. "templates/login.html")	Template* – handle template, NULL se non trovato
tpl_render()	Renderizza il template sostituendo i segnaposto {{KEY}} con i valori dell'array TplVar. Itera i frammenti pre-calcolati in O(n) senza riesaminare il sorgente.	tpl: const Template* – template da renderizzare dest: char* – buffer output destSize: size_t – capacità buffer vars: const TplVar* – array coppie chiave-valore nvars: int – numero di voci in vars	int – byte scritti, -1 su errore o overflow

Tabella 15. Funzioni dichiarate in *template.h*.

4.2.12 route_helpers.h

route_helpers.h

Helper interni condivisi tra *route_pages.c* e *route_api.c*. Non fanno parte dell'API pubblica. Includono la verifica della sessione, i primitivi di risposta (redirect, errori JSON/HTML, render template) e le funzioni di errore per-pagina.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
get_session_user()	Estrae e verifica il cookie di sessione dalla richiesta http.	req: const HttpRequest* – richiesta in ingresso u: User* – struct da popolare con i dati utente	bool – true se sessione valida, false se assente o scaduta
redirect()	Emette una risposta HTTP 302 verso l'URL indicato, con eventuale header Set-Cookie.	resp: HttpResponse* – risposta da popolare url: const char* – URL di destinazione cookie: const char* – valore Set-Cookie (NULL	void

		= ometti)	
<code>resp_json_error()</code>	Scrive un body JSON di errore con il codice di stato indicato.	resp: HttpResponse* status: int – codice HTTP (es. 400, 403, 404) msg: const char* – messaggio di errore	<code>void</code>
<code>resp_html_error()</code>	Scrive un errore in HTML con il codice di stato indicato.	resp: HttpResponse* status: int – codice HTTP msg: const char* – messaggio di errore	<code>void</code>
<code>resp_render_tpl()</code>	Renderizza un template per nome.	resp: HttpResponse* tplName: const char* – percorso template vars: TplVar* – variabili da iniettare nVars: int – numero di variabili	<code>bool</code> – true su successo, false se template non trovato (imposta 500)
<code>login_error()</code>	Renderizza login.html iniettando il messaggio di errore nel segnaposto {{ERROR_BLOCK}}.	resp: HttpResponse* msg: const char* – messaggio da visualizzare	<code>void</code>
<code>register_error()</code>	Renderizza register.html iniettando il messaggio di errore nel segnaposto {{ERROR_BLOCK}}.	resp: HttpResponse* msg: const char* – messaggio da visualizzare	<code>void</code>
<code>submit_error()</code>	Renderizza submit.html con tutte le variabili mappa e il messaggio di errore in {{ERROR_BLOCK}}.	resp: HttpResponse* u: const User* – utente autenticato (per USERNAME, CITY, coordinate mappa) msg:	<code>void</code>

		const char* – messaggio da visualizzare	
--	--	---	--

Tabella 16. Funzioni dichiarate in *route_helpers.h*.

4.2.13 http_utils.h

http_utils.h

Suite di utilità HTTP per parsing, serializzazione e sanitizzazione. Integra *picohttpparser* per il parsing zero-copy. Divisa in tre livelli: verifica/parsing stream, serializzazione risposta, estrazione parametri e sanitizzazione HTML.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
http_request_parse()	Analizza i byte grezzi di rete in una struct <i>HttpRequest</i> tramite <i>picohttpparser</i> (zero-copy: i campi puntano direttamente nel buffer <i>raw</i>).	raw: const char* – buffer di ricezione TCP rawLen: size_t – byte disponibili req: <i>HttpRequest*</i> – struct da popolare	bool – true se il parsing è completato, false altrimenti
http_request_header()	Cerca un header per nome e restituisce un puntatore al valore	req: const <i>HttpRequest*</i> name: const char* – nome header valueLen: size_t* – output lunghezza valore	const char* – puntatore al valore, NULL se assente
http_request_cookie()	Localizza e decodifica il valore di un cookie specifico dall'header <i>Cookie</i> .	req: const <i>HttpRequest*</i> name: const char* – nome cookie dest: char* – buffer output max: size_t – capacità	void

		buffer	
<code>http_is_request_path()</code>	Confronto esatto del path della richiesta.	req: const HttpRequest* path: const char* – path atteso	bool – true se corrispond e
<code>http_is_request_method()</code>	Confronto esatto del metodo HTTP della richiesta.	req: const HttpRequest* method: const char* – metodo atteso (es. "GET")	bool – true se corrispond e
<code>http_is_request_complete()</code>	Verifica che gli header completi (e il body per POST) siano arrivati nel buffer.	buf: const char* – buffer accumulato len: size_t – byte disponibili	bool – true se la richiesta è completa
<code>http_request_contains_keepalive()</code>	Verifica se l'header Connection specifica una connessione persistente.	req: const HttpRequest*	bool – true se keep-alive
<code>http_response_render()</code>	Serializza la struct HttpResponse.	resp: const HttpResponse * – risposta sorgente keepAlive: bool – flag connessione persistente out: char* – buffer output outMax: size_t – capacità buffer	int – byte totali scritti, -1 su overflow
<code>get_field()</code>	Estrae e decodifica il valore di un	src: const char* – body URL-encoded	void

	singolo campo da un body application/x-www-form-urlencoded.	paramName: const char* – nome campo (es. "username=") dest: char* – buffer output max: size_t – capacità buffer	
<code>html_escape()</code>	Escaping caratteri pericolosi HTML (<, >, &, ", ') per prevenire attacchi XSS.	src: const char* – stringa sorgente dest: char* – buffer output max: size_t – capacità buffer	<code>void</code>
<code>make_error_block()</code>	Avvolge un messaggio di errore in un blocco HTML stilizzato pronto per l'iniezione nel template.	msg: const char* – messaggio da formattare dest: char* – buffer output max: size_t – capacità buffer	<code>void</code>

Tabella 17. Funzioni dichiarate in `http_utils.h`.

4.2.14 server.h

server.h

Ciclo di vita del server: setup TCP libuv, gestione SIGINT/SIGTERM per lo shutdown, inizializzazione di tutti i sottosistemi. Espone hash_key() condivisa da tutte le istanze di Hash_Table.

Funzione	Scopo	Parametri in ingresso	Tipo di ritorno
<code>hash_key()</code>	Funzione di hashing per chiavi binarie arbitrarie. Il seed per-tabella previene attacchi di hash-flooding. Condivisa da tutte le	key: const void* – puntatore ai byte della chiave keySize: size_t – numero di byte da	<code>unsigned long</code> – valore hash

	Hash_Table del sistema.	hashare seed: unsigned long – seed per-tabella	
--	-------------------------	--	--

Tabella 18. Funzioni dichiarate in server.h.

4.2.15 Ulteriore documentazione

[Grafo delle dipendenze](#) - [Manuale d'uso](#) (generato con Doxygen)

5. Codifica

Questa sezione descrive gli aspetti implementativi più rilevanti del sistema, con particolare attenzione alle strutture dati personalizzate e agli algoritmi che ne caratterizzano l'architettura. Ogni funzione è presentata tramite pseudocodice, accompagnato da un'analisi delle scelte progettuali, dei vantaggi e dei trade-off.

5.1 Funzione di hashing — hash_key()

Tutte le istanze di Hash_Table del sistema (sessioni utente, rate limiter, cache geografica) condividono la stessa funzione di hashing dichiarata in server.h. La scelta di un'unica implementazione condivisa garantisce coerenza nelle proprietà di distribuzione tra le diverse tabelle e riduce la superficie di manutenzione.

L'algoritmo è una variante MurmurHash: opera byte per byte sulla chiave, mescolando ogni byte tramite XOR, moltiplicazione per una costante magica e shift. Le tre operazioni finali fuori dal ciclo (avalanche step) garantiscono che anche differenze minime nell'input producano hash radicalmente diversi — proprietà chiamata avalanche effect (effetto a valanga), fondamentale per una distribuzione uniforme dei bucket.

```

FUNCTION hash_key(key, keySize, seed)
    data ← CAST key TO array of unsigned char
    h ← seed
    FOR i ← 0 TO keySize - 1 DO
        h ← h XOR data[i]
        h ← h * MURMUR_MUL
        h ← h XOR (h >> 15)
    END FOR
    h ← h XOR (h >> 13)
    h ← h * MURMUR_FIN
    h ← h XOR (h >> 16)
    RETURN h
END FUNCTION

```

Pseudocodice 1. hash_key(): funzione di hashing MurmurHash.

Il parametro `seed` è generato per ogni tabella da `/dev/urandom` al momento della creazione, rendendo ogni istanza di Hash_Table non deterministica tra esecuzioni diverse. Questo previene attacchi di hash-flooding: un attaccante che conosce la funzione ma non il seed non può costruire

serie di chiavi che collidano sistematicamente sullo stesso bucket, causando degenerazione della tabella da $O(1)$ a $O(n)$.

5.2 Hash Table generica thread-safe

5.2.1 Inserimento — `ht_set()`

L'operazione di inserimento acquisisce il write lock esclusivo prima di qualsiasi modifica. Il controllo della chiave esistente avviene scorrendo la catena del bucket: se trovata, il valore viene aggiornato in-place liberando il vecchio buffer e allocandone uno nuovo. Se la chiave non esiste e il load factor supera 1.0, viene eseguito il resize prima dell'inserimento.

La scelta di inserire il nuovo nodo in testa alla catena (head insertion) anziché in coda è un'operazione in $O(1)$ senza dover percorrere la catena fino alla fine. Poiché la tabella non garantisce ordinamento interno, questa scelta non comporta svantaggi semantici. Il campo hash viene memorizzato nella Entry al momento della creazione per evitare il ricalcolo durante il successivo rehashing.

```
FUNCTION ht_set(table, key, keySize, value, valueSize)

    IF table IS NULL OR key IS NULL THEN
        RETURN 0
    END IF

    ACQUIRE WRITE LOCK(table.lock)
    h ← hashFunction(key, keySize, table.seed)
    index ← h MOD table.capacity

    FOR EACH entry e IN table.pool[index] DO
        IF keys_equal(e.key, e.keySize, key, keySize) THEN
            newValue ← ALLOCATE(valueSize)
            IF newValue IS NULL THEN GOTO ERROR END IF

            FREE(e.value)
            e.value ← newValue
            COPY value TO e.value
            e.size ← valueSize

            RELEASE LOCK(table.lock)
            RETURN 1
        END IF
    END FOR

    IF table.size + 1 ≥ table.capacity THEN
```

```

        IF resize(table) FAILS THEN GOTO ERROR END IF

        index ← h MOD table.capacity
    END IF

    newEntry ← CREATE_ENTRY(key, keySize, value, valueSize, h)
    IF newEntry IS NULL THEN GOTO ERROR END IF

    newEntry.next ← table.pool[index]
    table.pool[index] ← newEntry
    table.size ← table.size + 1

    RELEASE_LOCK(table.lock)
    RETURN 1

ERROR:
    RELEASE_LOCK(table.lock)
    RETURN 0

END FUNCTION

```

Pseudocodice 2. ht_set(): inserimento con write lock, aggiornamento in-place e resize.

5.2.2 Resize e rehashing — ht_resize()

La capacità viene raddoppiata e poi portata al primo numero primo superiore. I numeri primi come capacità riducono le collisioni da modulo quando le chiavi hanno pattern periodici (es. sequenze numeriche): un divisore primo garantisce che l'operazione modulo distribuisca uniformemente le chiavi anche in presenza di regolarità. Il rehashing riusa le Entry già allocate, azzerando l'array dei bucket e reinserendo ogni nodo con il nuovo indice calcolato dall'hash già memorizzato.

```

FUNCTION ht_resize(table)

    newCap ← next_prime(table->capacity * 2)
    newPool ← CALLOC(newCap, size_of(Entry*))

    IF newPool IS NULL THEN
        RETURN 0
    END IF

    FOR i ← 0 TO table->capacity - 1 DO
        e ← table->pool[i]

```

```

    WHILE e IS NOT NULL DO
        next ← e.next
        newIndex ← e.hash MOD newCap

        e.next ← newPool[newIndex]
        newPool[newIndex] ← e

        e ← next
    END WHILE

END FOR

FREE(table->pool)

table->pool ← newPool
table->capacity ← newCap

RETURN 1

END FUNCTION

```

Pseudocodice 3. ht_resize(): rehashing con hash cached e capacità prima.

5.2.3 Lettura concorrente — ht_get()

Le letture acquisiscono un read lock, consentendo a più thread di leggere simultaneamente dalla stessa tabella. La scelta del readers-writer lock rispetto a un mutex semplice è motivata dalla possibilità di far diventare il sistema multithreaded: più thread potrebbero leggere contemporaneamente dalla Hash table.

```

FUNCTION ht_get(table, key, keySize, destBuffer, destSize)

    IF table IS NULL OR key IS NULL OR destBuffer IS NULL THEN
        RETURN 0
    END IF

    ACQUIRE READ LOCK(table.lock)

    index ← hashFunction(key, keySize, table.seed) MOD table.capacity

```

```

FOR EACH entry e IN table.pool[index] DO
    IF keys_equal(e.key, e.keySize, key, keySize) THEN

        n ← MIN(e.size, destSize)
        COPY e.value INTO destBuffer (n bytes)

        RELEASE LOCK(table.lock)
        RETURN 1

    END IF
END FOR

RELEASE LOCK(table.lock)
RETURN 0

END FUNCTION

```

Pseudocodice 4. ht_get(): lettura con read lock per concorrenza.

5.3 Slab Allocator

Le connessioni TCP attive (struct ClientCtx) vengono gestite tramite uno slab allocator personalizzato. Con migliaia di connessioni al secondo, invocare malloc/free di sistema per ogni ClientCtx provocherebbe frammentazione della heap e pressione sul memory allocator globale. Lo slab pre-alloca grandi chunk contigui e li suddivide in slot di dimensione fissa, mantenendo alloc/free in O(1) puro senza alcuna ricerca.

5.3.1 Creazione di un chunk — create_chunk()

Ogni chunk è un unico blocco contiguo allocato con calloc. La free-list locale è costruita durante la creazione dei blocchi nell'area utente. Ogni slot contiene un back-pointer nascosto nei sizeof(void*) byte che lo precedono: questo permette a slab_pool_free() di risalire al Chunk proprietario in O(1) senza scorrere la lista dei chunk, che potrebbe crescere indefinitamente. La scelta di nascondere il back-pointer nell'header del blocco — invisibile al chiamante — mantiene l'API pulita e fa sì che i ClientCtx non debbano sapere nulla dello slab.

```

FUNCTION create_chunk(pool)

    dataSize ← pool->chunkCapacity * pool->internalBsize

    c ← CALLOC(1, SIZEOF(Chunk) + dataSize)

    IF c IS NULL THEN
        RETURN NULL
    END IF

```

```

END IF

c.capacity ← pool->chunkCapacity
c.usedCount ← 0

FOR i ← 0 TO c.capacity - 1 DO

    blockStart ← c.data + (i * pool->internalBsize)

    STORE (Chunk*)c INTO blockStart

    userArea ← blockStart + SIZEOF(pointer)

    STORE c.localFree INTO userArea
    c.localFree ← userArea

END FOR

RETURN c

END FUNCTION

```

Pseudocodice 5. create_chunk(): free-list con back-pointer nascosto.

5.3.2 Allocazione — slab_pool_alloc()

L'allocazione percorre la lista di chunk alla ricerca del primo con slot disponibili. Se nessuno ne ha, viene creato un nuovo chunk e preposto in testa. Il pop dalla free-list locale è una singola dereferenziazione e aggiornamento di puntatore. La memoria restituita non è inizializzata a 0: il chiamante (client_manager_alloc) azzerava esplicitamente i campi rilevanti, evitando il costo di calloc sull'intera struct per ogni connessione.

```

FUNCTION slab_pool_alloc(pool)

    IF pool IS NULL THEN
        RETURN NULL
    END IF

    chunk ← pool.chunksHead

    WHILE chunk IS NOT NULL AND chunk.localFree IS NULL DO

```

```

        chunk ← chunk.next
    END WHILE

    IF chunk IS NULL THEN
        chunk ← create_chunk(pool)
        IF chunk IS NULL THEN
            RETURN NULL
        END IF

        chunk.next ← pool.chunksHead

        IF pool.chunksHead IS NOT NULL THEN
            pool.chunksHead.prev ← chunk
        END IF

        pool.chunksHead ← chunk
    END IF

    userPtr ← chunk.localFree
    chunk.localFree ← DEREFERENCE(userPtr)
    chunk.usedCount ← chunk.usedCount + 1

    RETURN userPtr

END FUNCTION

```

Pseudocodice 6. slab_pool_alloc(): pop dalla free-list locale in $O(1)$.

5.3.3 Deallocazione — slab_pool_free()

La deallocazione usa il back-pointer nascosto (nei sizeof(void*) byte precedenti l'area utente) per identificare il Chunk proprietario in $O(1)$, poi reinserisce lo slot nella free-list locale con un push. Se il chunk diventa completamente vuoto viene liberato, tranne l'ultimo rimasto: questo evita il pattern patologico in cui un server a basso carico alterna continuamente alloc/free dello stesso slot, creando un ciclo di malloc/free a livello di chunk. Tenere almeno un chunk allocato mantiene il costo ammortizzato costante e evita il thrashing.

```

FUNCTION slab_pool_free(pool, ptr)

    IF ptr IS NULL THEN
        RETURN
    END IF

```



```

END IF

chunk ← DEREFERENCE(ptr - SIZEOF(pointer))

STORE chunk.localFree INTO ptr
chunk.localFree ← ptr
chunk.usedCount ← chunk.usedCount - 1

IF chunk.usedCount == 0 AND (chunk.next IS NOT NULL OR chunk.prev IS NOT
NULL) THEN

    IF chunk.prev IS NOT NULL THEN
        chunk.prev.next ← chunk.next
    END IF

    IF chunk.next IS NOT NULL THEN
        chunk.next.prev ← chunk.prev
    END IF

    IF pool.chunksHead == chunk THEN
        pool.chunksHead ← chunk.next
    END IF

    FREE(chunk)

END IF

END FUNCTION

```

Pseudocodice 7. slab_pool_free(): push $O(1)$ con back-point.

5.4 Gestione connessioni — lista doppiamente concatenata

Il client manager mantiene tutte le connessioni TCP attive in una lista doppiamente concatenata globale. La scelta della lista doppia anziché semplice è motivata dall'operazione di unlink: con una lista semplice sarebbe necessario percorrere la lista dal capo per trovare il predecessore ($O(n)$); con i puntatori prev/next la rimozione avviene aggiornando i due nodi adiacenti in $O(1)$, indipendentemente dalla dimensione della lista e dalla posizione del nodo.

5.4.1 Inserimento in testa — client_manager_link()

```

FUNCTION client_manager_link(ctx)

```

```

    ctx.next ← ptrActiveClientsHead
    ctx.prev ← NULL

    IF ptrActiveClientsHead IS NOT NULL THEN
        ptrActiveClientsHead.prev ← ctx
    END IF

    ptrActiveClientsHead ← ctx
    activeClientsCount ← activeClientsCount + 1

END FUNCTION

```

Pseudocodice 8. client_manager_link(): inserimento $O(1)$ in testa.

L'inserimento in testa è scelto perché non richiede un puntatore alla coda (che richiederebbe aggiornamento ad ogni inserimento/rimozione) e perché l'ordine delle connessioni non è semanticamente rilevante: lo shutdown le chiude comunque tutte.

5.4.2 Rimozione — client_manager_unlink()

La rimozione distingue tre casi: nodo in testa (aggiorna il puntatore globale), nodo con predecessore (aggiorna next del predecessore), nodo con successore (aggiorna prev del successore). I casi non si escludono a vicenda: un nodo può avere sia predecessore sia successore, e aggiornare entrambi è sempre corretto.

```

FUNCTION client_manager_unlink(ctx)

    IF ctx.prev IS NOT NULL THEN
        ctx.prev.next ← ctx.next
    ELSE
        ptrActiveClientsHead ← ctx.next
    END IF

    IF ctx.next IS NOT NULL THEN
        ctx.next.prev ← ctx.prev
    END IF

    ctx.next ← NULL
    ctx.prev ← NULL

    activeClientsCount ← activeClientsCount - 1

END FUNCTION

```

Pseudocodice 9. client_manager_unlink(): rimozione O(1).

5.4.3 Chiusura di tutte le connessioni — client_manager_close_all()

Durante lo shutdown tutte le connessioni devono essere chiuse. Il punto critico è che il callback `close_client` chiama internamente `client_manager_unlink`, che sovrascrive `curr->next`: senza precauzione, il puntatore al successore andrebbe perso. La soluzione è salvare il puntatore al successore prima di invocare il callback — pattern classico.

```
FUNCTION client_manager_close_all(fn)

    curr ← ptrActiveClientsHead

    WHILE curr IS NOT NULL DO

        next ← curr.next

        // fn may unlink curr and modify curr.next, so we cache next first
        CALL fn(curr)

        curr ← next

    END WHILE

END FUNCTION
```

Pseudocodice 10. client_manager_close_all().

5.5 Layer di rete — ciclo di lettura libuv

Il cuore del layer di rete è il callback `on_read()`, invocato da libuv ogni volta che dati TCP diventano disponibili su una connessione. Il server adotta un'architettura event-driven a singolo thread: `on_read()` non blocca mai e restituisce il controllo al loop libuv non appena ha finito, permettendo la gestione concorrente di migliaia di connessioni senza thread aggiuntivi.

5.5.1 Allocazione lazy del buffer — on_buffer_alloc()

Il buffer di ricezione viene allocato la prima volta che arrivano dati su una connessione, non al momento dell'accettazione. Questo è fondamentale per le connessioni keep-alive inattive che richiederebbero diversi KB di buffering, senza utilizzarli a pieno. Nelle chiamate successive viene restituita la porzione inutilizzata del buffer già allocato.

```
FUNCTION on_buffer_alloc(handle, suggestedSize, buf)
```

```

    ctx ← handle.data

    IF ctx.buffer IS NULL THEN
        ctx.buffer ← ALLOCATE(CLIENT_BUFFER_SIZE)

        IF ctx.buffer IS NULL THEN
            buf.base ← NULL
            buf.len ← 0
            RETURN
        END IF
    END IF

    remaining ← CLIENT_BUFFER_SIZE - 1 - ctx.totalRead

    IF remaining > 0 THEN
        buf.base ← ctx.buffer + ctx.totalRead
    ELSE
        buf.base ← NULL
    END IF

    buf.len ← remaining

END FUNCTION

```

Pseudocodice 11. on_buffer_alloc(): allocazione lazy.

5.5.2 Gestione della lettura e dispatch — on_read()

on_read() accumula i byte in arrivo fino al completamento della richiesta HTTP, poi esegue il parsing, controlla il rate limiting, passa la richiesta dal route handler e invia la risposta. La chiave è che ogni fase può restituire il controllo a libuv senza processare la richiesta: se i dati non sono ancora completi, on_read() ritorna semplicemente senza fare nulla, e libuv richiamerà il callback quando arriveranno altri byte.

```

FUNCTION on_read(stream, nread, buf)

    ctx ← stream.data

    IF nread < 0 THEN
        close_client(ctx)
        RETURN
    END IF

```

```

END IF

IF nread = 0 THEN
    RETURN
END IF

ctx.totalRead ← ctx.totalRead + nread
ctx.buffer[ctx.totalRead] ← '\0'

IF request_is_complete(ctx.buffer, ctx.totalRead) = FALSE THEN
    RETURN
END IF

STOP_READING(stream)
STOP_TIMER(ctx.timer)

ip ← GET_PEER_IP(ctx.handle)

IF DEBUG_RATE_LIMIT AND rate_limit_check(ip) = FALSE THEN

    response ← CREATE_RESPONSE(429, "Too Many Requests")
    send_response(ctx, response, FALSE)

    RETURN
END IF

req ← EMPTY_REQUEST()

IF parse_request(ctx.buffer, ctx.totalRead, req) FAILS THEN

    response ← CREATE_RESPONSE(400, "<h1>400 Bad Request</h1>")
    send_response(ctx, response, FALSE)

    RETURN
END IF

keepAlive ← request_contains_keepalive(req)

```

```

    response ← CREATE_RESPONSE(200, global_response_buffer)

    handle_request(req, response)

    send_response(ctx, response, keepAlive)

END FUNCTION

```

Pseudocodice 12. on_read(): parsing e dispatch event-driven.

5.6 Gestione delle sessioni

5.6.1 Creazione del token — session_create()

Il token di sessione è una stringa esadecimale casuale a 32 caratteri. Il ciclo di collision avoidance rigenera il token finché non trova uno slot libero nella hash table.

```

FUNCTION session_create(user, outToken)

    IF s_sessions IS NULL OR user IS NULL OR outToken IS NULL THEN
        RETURN FALSE
    END IF

    dummy ← EMPTY_USER()

    FOR tries ← 0 TO SESSION_CREATE_MAX_TRIES - 1 DO

        token ← GENERATE_TOKEN(TOKEN_HEX_LEN)

        IF session_verify(token, dummy) = FALSE THEN
            BREAK
        END IF

    END FOR

    IF tries = SESSION_CREATE_MAX_TRIES THEN
        RETURN FALSE
    END IF

    s.user ← user

```

```

s.createdAt ← CURRENT_TIME()

IF ht_set(s_sessions, token, s) FAILS THEN
    RETURN FALSE
END IF

COPY token INTO outToken

RETURN TRUE

END FUNCTION

```

Pseudocodice 13. session_create(): token con collision avoidance.

5.6.2 Verifica con eviction lazy — session_verify()

Le sessioni scadute non vengono eliminate da un thread di pulizia dedicato: vengono rimosse al primo accesso successivo alla scadenza (lazy eviction). Questo approccio è preferito perché il server è single-threaded e un thread separato richiederebbe sincronizzazione aggiuntiva. Il costo è che le sessioni scadute occupano memoria fino al primo accesso, ma con SESSION_MAX_AGE = 24h e MAX_CLIENTS connessioni massime, il consumo è limitato e prevedibile.

```

FUNCTION session_verify(token, outUser)

    IF s_sessions IS NULL OR token IS NULL THEN
        RETURN FALSE
    END IF

    IF ht_get(s_sessions, token, session) FAILS THEN
        RETURN FALSE
    END IF

    IF CURRENT_TIME() - session.createdAt > SESSION_MAX_AGE THEN

        DELETE_SESSION(s_sessions, token)

        RETURN FALSE

    END IF

    IF outUser IS NOT NULL THEN

```

```

        outUser ← session.user

    END IF

    RETURN TRUE

END FUNCTION

```

Pseudocodice 14. session_verify(): eviction lazy senza thread dedicato.

5.7 Rate limiter a finestra scorrevole

Il rate limiter protegge il server da abusi tracciando il numero di richieste per IP al secondo. L'algoritmo usa due soli contatori per IP anziché una lista di timestamp: il contatore della finestra corrente e quello della finestra precedente. Il tasso istantaneo viene stimato

Il vantaggio rispetto alla finestra fissa è l'eliminazione dei burst: con una finestra fissa un client potrebbe inviare RATE_LIMIT_RPS richieste nell'ultimo millisecondo di una finestra e RATE_LIMIT_RPS nel primo della successiva, totalizzando 2× il limite in meno di 100ms. L'interpolazione lineare pesa il contributo della finestra precedente proporzionalmente al tempo già trascorso nella finestra corrente, smorzando questi picchi.

```

FUNCTION rate_limit_check(ip)

    IF ip IS NULL OR ip IS EMPTY OR rateTable IS NULL THEN
        RETURN 1
    END IF

    IF rateTable.size > 10000 THEN

        DESTROY(rateTable)

        rateTable ← CREATE_HASH_TABLE(1024)

        IF rateTable IS NULL THEN
            RETURN 1
        END IF

    END IF

    entry ← EMPTY_RATE_ENTRY()
    now ← CURRENT_TIME()

    GET(rateTable, ip, entry)

```



```

IF now - entry.windowStartTime ≥ 1 THEN
    entry.countPrev ← entry.countCurr
    entry.countCurr ← 0
    entry.windowStartTime ← now
END IF

elapsed ← now - entry.windowStartTime

estimated ← entry.countPrev × (1 - elapsed)
           + entry.countCurr

allowed ← (estimated < RATE_LIMIT_RPS)

IF allowed THEN
    entry.countCurr ← entry.countCurr + 1
END IF

SET(rateTable, ip, entry)

RETURN allowed

END FUNCTION

```

Pseudocodice 15. rate_limit_check(): finestra scorrevole con interpolazione lineare.

5.8 Cache JSON in-memory per i report

5.8.1 Lookup con eviction lazy — cache_find()

La cache è un array statico di 32 slot (`g_cache[32]`). Il TTL di 5 secondi è scelto come compromesso tra freschezza dei dati (le segnalazioni cambiano raramente in una finestra di 5s) e riduzione del carico su SQLite.

```

FUNCTION cache_find(city, userId, isOperator, isArchived)

    now ← CURRENT_TIME()

    FOR i ← 0 TO CACHE_MAX_ENTRIES - 1 DO

        e ← g_cache[i]

```

```

    IF e.cachedAt == 0 THEN
        CONTINUE
    END IF

    IF now - e.cachedAt > CACHE_TTL_SECONDS THEN
        e.cachedAt ← 0
        CONTINUE
    END IF

    IF e.isOperator == isOperator AND e.isArchived == isArchived
        AND string_compare(e.city, city, CITY_LEN)
        AND e.userId == userId THEN
        RETURN i
    END IF

END FOR

RETURN -1

END FUNCTION

```

Pseudocodice 16. cache_find(): scansione lineare report cache.

5.8.2 Inserimento LRU-approx — cache_store()

L'inserimento cerca uno slot libero e, se non lo trova, individua il candidato da sostituire (l'entry con timestamp più vecchio) in un'unica passata sull'array. Una LRU esatta richiederebbe una lista concatenata separata per tracciare l'ordine di accesso aggiungendo ulteriore complessità. L'approssimazione è sufficiente perché l'obiettivo non è massimizzare l'hit rate ma evitare di sovrascrivere entry ancora valide quando possibile.

```

FUNCTION cache_store(city, userId, isOperator, isArchived, json, jsonLen)

    IF jsonLen ≥ CACHE_JSON_MAX THEN
        RETURN
    END IF

    slot ← -1
    oldestIdx ← 0
    oldestTime ← g_cache[0].cachedAt

```

```

FOR i ← 0 TO CACHE_MAX_ENTRIES - 1 DO

    IF g_cache[i].cachedAt == 0 THEN
        slot ← i
        BREAK
    END IF

    IF g_cache[i].cachedAt < oldestTime THEN
        oldestTime ← g_cache[i].cachedAt
        oldestIdx ← i
    END IF

END FOR

IF slot == -1 THEN
    slot ← oldestIdx
END IF

e ← g_cache[slot]

COPY city INTO e.city (max CITY_LEN - 1 chars)
e.city[CITY_LEN - 1] ← '\0'

e.userId ← userId
e.isOperator ← isOperator
e.isArchived ← isArchived
e.jsonLen ← jsonLen

COPY json INTO e.json (jsonLen + 1 bytes)

e.cachedAt ← CURRENT_TIME()

END FUNCTION

```

Pseudocodice 17. cache_store(): inserimento in una passata con LRU approssimata.

5.8.3 Lettura con integrazione cache — report_get_active_json()

La funzione integra lookup sulla cache, selezione differenziata della query SQL per ruolo e reinserimento del risultato in cache. I risultati vuoti non vengono memorizzati: un array vuoto potrebbe essere uno stato transitorio (es. nessuna segnalazione assegnata all'operatore in questo momento)

```
FUNCTION report_get_active_json(buf, max, userId, city, isOperator)

    idx ← cache_find(city, userId, isOperator, false)

    IF idx ≥ 0 THEN
        COPY g_cache[idx].json INTO buf (g_cache[idx].jsonLen + 1 bytes)
        RETURN g_cache[idx].jsonLen
    END IF

    IF isOperator THEN
        c ← db_cursor_open(
            "SELECT ... FROM reports "
            "WHERE assigned_to = ? AND status IN (1, 2) "
            "ORDER BY assigned_at DESC;",
            userId
        )
    ELSE
        c ← db_cursor_open(
            "SELECT ... FROM reports "
            "WHERE author_id = ? AND status < 3 "
            "ORDER BY created_at DESC;",
            userId
        )
    END IF

    len ← cursor_to_json_array(c, buf, max)

    IF len > 0 THEN
        cache_store(city, userId, isOperator, false, buf, len)
    END IF

    RETURN len
```

```
END FUNCTION
```

Pseudocodice 18. report_get_active_json():cache→DB→serializzazione.

5.9 Ciclo di vita delle segnalazioni

Tutte le transizioni di stato sono implementate con UPDATE SQL che includono una clausola WHERE composta che funge da guardia. Se la condizione non è soddisfatta, db_changes() restituisce 0 (guard failed)..

5.9.1 Assegnazione admin — report_assign()

```
FUNCTION report_assign(reportId, operatorId)

    rc ← db_exec(
        "UPDATE reports "
        "SET status = 1, assigned_to = ?, assigned_at = ? "
        "WHERE id = ? AND status = 0 AND assigned_to IS NULL;",
        operatorId,
        CURRENT_TIME(),
        reportId
    )

    IF rc ≠ 0 THEN
        RETURN -1
    END IF

    IF db_changes() == 0 THEN
        RETURN 0
    END IF

    r ← EMPTY_REPORT()

    IF report_get_by_id(reportId, r) THEN
        report_cache_invalidate_city(r.city, r.authorId)
    END IF

    RETURN 1

END FUNCTION
```

Pseudocodice 19. report_assign(): guardia WHERE status=0 AND assigned_to IS NULL.

5.9.2 Accettazione operatore — report_accept()

```
FUNCTION report_accept(reportId, operatorId)

    rc ← db_exec(
        "UPDATE reports "
        "SET status = 2 "
        "WHERE id = ? AND assigned_to = ? AND status = 1;",
        reportId,
        operatorId
    )

    IF rc ≠ 0 THEN
        RETURN -1
    END IF

    IF db_changes() == 0 THEN
        RETURN 0
    END IF

    r ← EMPTY_REPORT()

    IF report_get_by_id(reportId, r) THEN
        report_cache_invalidate_city(r.city, r.authorId)
    END IF

    RETURN 1

END FUNCTION
```

Pseudocodice 20. report_accept(): ASSIGNED → IN_PROGRESS con verifica proprietà.

5.9.3 Rifiuto operatore — report_reject()

In caso di rifiuto, la segnalazione torna a STATUS_ACTIVE e `assigned_to` viene azzerato a NULL, rendendola disponibile per una nuova assegnazione dall'admin:

```
FUNCTION report_reject(reportId, operatorId)

    rc ← db_exec(
        "UPDATE reports "
```

```

        "SET status = 0, assigned_to = NULL, assigned_at = NULL "
        "WHERE id = ? AND assigned_to = ? AND status = 1;",
        reportId,
        operatorId
    )

    IF rc ≠ 0 THEN
        RETURN -1
    END IF

    IF db_changes() == 0 THEN
        RETURN 0
    END IF

    r ← EMPTY_REPORT()

    IF report_get_by_id(reportId, r) THEN
        report_cache_invalidate_city(r.city, r.authorId)
    END IF

    RETURN 1

END FUNCTION

```

Pseudocodice 21. report_reject(): ASSIGNED → ACTIVE con azzeramento assegnazione.

5.9.4 Feedback del cittadino — report_set_feedback()

Tre condizioni nella stessa WHERE garantiscono le invarianti in un'unica operazione atomica: solo l'autore, solo su RESOLVED, solo una volta (`feedback IS NULL`):

```

FUNCTION report_set_feedback(reportId, authorId, stars)

    IF stars < 1 OR stars > 5 THEN
        RETURN -1
    END IF

    rc ← db_exec(
        "UPDATE reports "

```

```

        "SET feedback = ? "

        "WHERE id = ? AND author_id = ? AND status = 3 AND feedback IS NULL;",
        stars,
        reportId,
        authorId
    )

    IF rc ≠ 0 THEN
        RETURN -1
    END IF

    IF db_changes() == 0 THEN
        RETURN 0
    END IF

    r ← EMPTY_REPORT()

    IF report_get_by_id(reportId, r) THEN
        report_cache_invalidate_city(r.city, r.authorId)
    END IF

    RETURN 1

END FUNCTION

```

Pseudocodice 22. report_set_feedback(): guardia tripla per feedback unico.

5.10 Autenticazione utente— user_authenticate()

La funzione di autenticazione applica un unico schema di hashing delle password basato su **salt per utente**, calcolando: `hash=SHA256(salt||password)`.

Il salt, memorizzato nel database insieme ai dati dell'utente, viene concatenato alla password in chiaro prima dell'hashing. L'uso di un salt univoco impedisce che utenti con la stessa password producano hash identici e riduce l'efficacia di attacchi basati su rainbow table o precomputazione.

Il confronto finale usa `constant_time_compare()` anziché `strcmp()` che si interrompe al primo byte diverso, rendendo misurabile il numero di byte iniziali che coincidono (timing attack). Un confronto a tempo costante confronta sempre tutti i byte indipendentemente dal punto di differenza, eliminando questa informazione dall'osservatore esterno.

```

FUNCTION user_authenticate(username, plainPassword, out)

```



```

    IF username IS NULL OR plainPassword IS NULL OR out IS NULL THEN
        RETURN FALSE
    END IF

    c ← db_cursor_open(
        "SELECT id, username, password_hash, role, city, salt "
        "FROM users WHERE username = ?;",
        username
    )

    found ← db_cursor_next(c)

    IF found THEN
        cursor_to_user(c, out)
    END IF

    db_cursor_close(c)

    IF found = FALSE THEN
        RETURN FALSE
    END IF

    loginHash ← HASH_PASSWORD(out.salt, plainPassword)

    RETURN constant_time_compare(out.passwordHash, loginHash)

END FUNCTION

FUNCTION user_get_by_id(id, out)

    IF out IS NULL THEN
        RETURN FALSE
    END IF

    c ← db_cursor_open(
        "SELECT id, username, password_hash, role, city, salt "

```

```

        "FROM users WHERE id = ?;",
        id
    )

    found ← db_cursor_next(c)

    IF found THEN
        cursor_to_user(c, out)
    END IF

    db_cursor_close(c)

    RETURN found

END FUNCTION

```

Pseudocodice 24. user_authenticate(): doppio schema con retrocompatibilità e timing-safe compare.

5.11 Validazione geografica — geo_contains() e build_map_vars()

Il modulo geografico espone due funzionalità principali: la validazione delle coordinate di una segnalazione rispetto al comune dell'utente (geo_contains) e la generazione delle variabili mappa per i template HTML che usano il framework Leaflet. Entrambe lavorano sulla hash table interna indicizzata per nome comune, popolata al caricamento del GeoJSON ISTAT.

5.11.1 Test di appartenenza al bounding box — geo_contains()

Il test usa un bounding box axis-aligned (rettangolo allineato agli assi latitudine/longitudine). Questa è un'approssimazione del poligono reale del comune: il bounding box è sempre più grande del comune, quindi alcune coordinate esterne al comune ma interne al box potrebbero passare la validazione. Questo trade-off è accettabile perché le coordinate errate di qualche chilometro fuori dal confine reale non causano danni funzionali, e un test poligonale esatto richiederebbe algoritmi molto più complessi e dataset più pesanti.

```

FUNCTION geo_lookup(city, out)

    IF cityTable IS NULL OR city IS NULL THEN
        RETURN FALSE
    END IF

    RETURN ht_get(cityTable, city, out)

END FUNCTION

```

```

FUNCTION geo_contains(geo, lat, lon)

    RETURN (lat ≥ geo.latMin AND lat ≤ geo.latMax)
        AND (lon ≥ geo.lonMin AND lon ≤ geo.lonMax)

END FUNCTION

```

Pseudocodice 25. geo_lookup() e geo_contains(): lookup $O(1)$ e test AABB.

5.11.2 Costruzione variabili mappa — build_map_vars()

La funzione prepara le stringhe che verranno iniettate nei template HTML per centrare e limitare la mappa Leaflet sul comune dell'utente.

```

FUNCTION build_map_vars(cityName, mv)

    geo ← EMPTY_CITY_GEO()

    IF geo_lookup(cityName, geo) THEN

        mv.lat ← FORMAT("%.6f", geo.centroidLat)
        mv.lon ← FORMAT("%.6f", geo.centroidLon)

        mv.bounds ← FORMAT(
            "[[%.6f,%.6f],[%.6f,%.6f]]",
            geo.latMin, geo.lonMin,
            geo.latMax, geo.lonMax
        )

    ELSE

        mv.lat ← "41.9"
        mv.lon ← "12.5"
        mv.bounds ← "null"

    END IF

END FUNCTION

```

Pseudocodice 26. build_map_vars(): variabili Leaflet con fallback a Roma.

5.12 Template engine — pre-parsing e rendering

5.12.1 Pre-parsing dei frammenti — `parse_fragments()`

Il parser scansiona il sorgente del template esattamente due volte. La prima passata conta i frammenti senza allocare memoria; la seconda alloca l'array della dimensione esatta e lo riempie. Questo two-pass approach evita il realloc incrementale (che richiederebbe copiare l'array ad ogni crescita) e garantisce che l'allocazione sia sempre della dimensione minima necessaria. I puntatori `ptr` dei frammenti LITERAL (testo statico) puntano direttamente nel buffer sorgente senza copie aggiuntive.

```
static int parse_fragments(Template *tpl) {
    const char *src = tpl->src;
    const char *end = src + tpl->srcLen;
    int count = 0;

    // First pass: count fragments.
    const char *cur = src;
    while (cur < end) {
        const char *open = find_open(cur, end);
        if (!open) {
            if (cur < end) count++; // trailing literal after the last variable
            break;
        }
        if (open > cur) count++; // literal before this '{{'
        count++;                // variable fragment
        const char *ke = find_close(open + 2, end);
        cur = (ke < end) ? ke + 2 : end;
    }

    tpl->fragments = malloc((count ? count : 1) * sizeof(Fragment));
    if (!tpl->fragments) return -1;
    tpl->fragCount = 0;

    // Second pass: populate fragments.
    cur = src;
    while (cur < end) {
        const char *open = find_open(cur, end);
        if (!open) {
            if (cur < end)
```

```

        tpl->fragments[tpl->fragCount++] =
            (Fragment){ FRAG_LITERAL, cur, (size_t)(end - cur) };
        break;
    }
    if (open > cur)
        tpl->fragments[tpl->fragCount++] =
            (Fragment){ FRAG_LITERAL, cur, (size_t)(open - cur) };

    const char *ks = open + 2;
    const char *ke = find_close(ks, end);
    tpl->fragments[tpl->fragCount++] =
        (Fragment){ FRAG_VARIABLE, ks, (size_t)(ke - ks) };
    cur = (ke < end) ? ke + 2 : end;
}

return 0;
}

```

Pseudocodice 27. parse_fragments(): pre-parsing in doppia passata senza realloc.

5.12.2 Rendering — tpl_render()

Il rendering itera i frammenti pre-calcolati in $O(n)$ senza mai riesaminare il sorgente. Per i FRAG_LITERAL (testo statico) la copia è diretta (puntatore già nel sorgente); per i FRAG_VARIABLE (placeholder) viene eseguita una ricerca lineare sull'array TplVar[] passato dal chiamante. Il numero di variabili per template è piccolo (al massimo 6), quindi la ricerca lineare è molto veloce. Il controllo di overflow precede ogni copia, garantendo che il buffer di destinazione non venga mai sovrascritto.

```

FUNCTION tpl_render(tpl, dest, destSize, vars, nvars)

    IF tpl IS NULL OR dest IS NULL OR destSize == 0 THEN
        RETURN -1
    END IF

    written ← 0

    FOR i ← 0 TO tpl.fragCount - 1 DO

        f ← tpl.fragments[i]

```

```

    IF f.type == FRAG_LITERAL THEN
        data ← f.ptr
        len ← f.len
    ELSE
        data ← lookup(f.ptr, f.len, vars, nvars)
        len ← LENGTH(data)
    END IF

    IF written + len + 1 > destSize THEN
        dest[written] ← '\0'
        RETURN -1
    END IF

    COPY data INTO dest[written]
    written ← written + len

END FOR

dest[written] ← '\0'

RETURN written

END FUNCTION

```

Pseudocodice 28. `tpl_render()`: $O(n \text{ frammenti})$ con lookup lineare variabili.

6. Test

Questa sezione raccoglie tutti i test condotti sul sistema Segnalazioni Urbane, organizzati in quattro aree: test dei requisiti funzionali, test di sicurezza, test di performance e test di copertura del codice. I casi di test includono sia scenari di successo sia scenari di fallimento — inclusi bug realmente riscontrati durante lo sviluppo, poi corretti e riverificati.

La colonna Risultato ottenuto è colorata in verde per i test superati e in rosso per quelli falliti. I test falliti documentano bug reali che sono stati poi risolti nel caso di test immediatamente successivo.

6.1 Test dei requisiti funzionali

I test funzionali verificano che ogni requisito (R01–R19) produca il comportamento descritto nei casi d'uso. Sono stati eseguiti 55 test tramite richieste HTTP dirette al server in esecuzione locale.

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
R01, R03, R05 — Login e accesso al sistema						
R01	T01.1	Pagina login	GET / senza sessione attiva: il server deve rispondere con la pagina HTML di login.	GET /	HTTP 200, HTML con form login	HTTP 200, form login visualizzato
R01	T01.2	Redirect se già loggato	GET / con cookie di sessione valido: reindirizzamento a /home senza mostrare il login.	GET / + cookie valido	HTTP 302 → /home	HTTP 302 Location: /home
R03	T01.3	Login valido cittadino	POST /login con credenziali corrette di un cittadino.	username=mario, password=abc123	HTTP 302 → /home, Set-Cookie token 32 char	HTTP 302, token impostato correttamente
R0	T0	Login	POST /login	username=luca_op,	HTTP 302	HTTP

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
3	1.4	valido operatore	con credenziali corrette di un operatore.	password=pass99	→ /home, dashboard operatore	302, dashboar d operator e caricata
R0 3	T0 1.5	Login valido admin	POST /login con credenziali corrette di un amministrat ore.	username=admin_ba, password=sec!1	HTTP 302 → /home, dashboard admin	HTTP 302, dashboar d admin caricata
R0 1	T0 1.6	Credenz iali errate	POST /login con password sbagliata: deve restare sulla pagina di login con messaggio di errore.	username=mario, password=SBAGLIATA	HTTP 200, messaggio di errore inline	HTTP 200, «Userna me o passwor d errati» visibile
R0 1	T0 1.7	Userna me inesiste nte	POST /login con username non registrato.	username=chichessia	HTTP 200, messaggio di errore	HTTP 200, errore mostrato
R0 5	T0 1.8	Dashbo ard per ruolo	GET /home con sessione operatore: carica operator_ma p.html, non citizen_hom e.html.	GET /home + sessione operatore	HTTP 200, template operator_map .html	HTTP 200, mappa operator e caricata
R02, R03 — Registrazione nuovo utente						

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
R0 2	T0 2.1	Registra zione valida	POST /register con dati validi per un nuovo cittadino.	username=gio, password=pass77, city=Bari, role=0	HTTP 302 → /, utente salvato nel DB	HTTP 302, utente inserito corretta mente
R0 2	T0 2.2	Userna me duplicat o	POST /register con username già presente nel database.	username=mario (già esiste)	HTTP 200, «Username già in uso»	HTTP 200, errore usernam e duplicato
R0 2	T0 2.3	Passwor d troppo corta	POST /register con password di 4 caratteri (minimo 6).	password=abc	HTTP 200, errore lunghezza password	HTTP 200, «Passwo rd deve avere almeno 6 caratteri»
R0 2	T0 2.4	Comune non riconosc iuto	POST /register con comune non presente nel dataset ISTAT.	city=CittàFantasma	HTTP 200, «Comune non riconosciuto»	HTTP 200, errore comune visualizz ato
R0 2	T0 2.5	Second o admin per città	POST /register ruolo admin per città che ha già un amministrat ore.	city=Bari, role=2 (admin)	HTTP 200, «Esiste già un'amministra zione»	HTTP 200, errore unicità admin
R0 2	T0 2.6	Campo obbligat orio mancant e	POST /register senza il campo city.	city= (vuoto)	HTTP 200, «Tutti i campi sono obbligatori»	HTTP 200, errore campi mostrato
R04 — Logout e invalidazione sessione						
R0 4	T0 3.1	Logout con session e valida	GET /logout con cookie valido: token rimosso e	GET /logout + cookie valido	HTTP 302 → /, Set- Cookie Max-	HTTP 302, cookie eliminato

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
			cookie cancellato.		Age=0	dal browser
R0 4	T0 3.2	Logout senza cookie	GET /logout senza cookie: reindirizzam ento senza errori (idempotent e).	GET /logout (no cookie)	HTTP 302 → /	HTTP 302 senza errori
R0 4	T0 3.3	Token non riutilizza bile	Dopo il logout, il token deve essere rifiutato da GET /home.	GET /home + token invalidato	HTTP 302 → /	HTTP 302, rimandat o al login
R06, R07 — Inserimento nuova segnalazione						
R0 6	T0 4.1	Pagina / submit	GET /submit con sessione cittadino: form con mappa centrata sul comune.	GET /submit + sessione cittadino	HTTP 200, template submit.html con MAP_LAT/LO N	HTTP 200, mappa caricata corretta mente
R0 6	T0 4.2	Redirect operator e	GET /submit con sessione operatore: reindirizzam ento a /home.	GET /submit + sessione operatore	HTTP 302 → /home	HTTP 302, operator e reindirizz ato
R0 6	T0 4.3	Segnala zione valida	POST /submit con tutti i campi validi e coordinate interne al comune.	lat=41.12, lon=16.87, category=Buche stradali, description=Buca profonda	HTTP 302 → /home, record inserito	HTTP 302, segnalazi one salvata status=0
R0 7	T0 4.4	Coordin ate fuori comune	POST /submit con coordinate	lat=45.00, lon=9.00 (fuori Bari)	HTTP 200, errore	HTTP 200, «Le coordina

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
			esterne al bounding box del comune.		bounding box	te non appartengono alla tua città»
R06	T04.5	Descrizione mancante	POST /submit senza campo description.	description= (vuoto)	HTTP 200, «La descrizione è obbligatoria»	HTTP 200, errore descrizione mostrato
R06	T04.6	Categoria assente	POST /submit senza campo category: assegnata automaticamente «Altro».	category= (vuoto)	HTTP 302 → /home, category='Altro' nel DB	HTTP 302, categoria 'Altro' salvata
R06	T04.7	Descrizione con accapo (BUG)	POST /submit con description contenente \n: l'URL-encoding %0A causava troncamento del body. Il parser si fermava all'accapo e perdeva i campi successivi (lat, lon mancanti).	description=Buca%0Aprofonda (con accapo)	HTTP 302 → /home, segnalazione salvata con descrizione intera	ERRORE : lat e lon risultavano 0.0, segnalazione inserita con coordinate nulle
R06	T04.8	Descrizione con accapo (FIX)	Dopo la correzione di get_field() per gestire %0A come carattere valido nel valore del campo, lo	description=Buca%0Aprofonda (con accapo)	HTTP 302 → /home, descrizione intera salvata, lat/lon corretti	HTTP 302, descrizione salvata con \n, coordinate

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
			stesso input deve essere accettato correttament e.			corrette
R08, R09 — Visualizzazione segnalazioni						
R08	T05.1	Segnalazioni attive cittadino	GET /api/reports/active con sessione cittadino: proprie segnalazioni non risolte.	GET /api/reports/active + sessione cittadino	HTTP 200, JSON array status 0–1	HTTP 200, array corretto
R08	T05.2	Segnalazioni attive operatore	GET /api/reports/active con sessione operatore: segnalazioni IN_PROGRESS assegnate.	GET /api/reports/active + sessione operatore	HTTP 200, JSON status=2, assigned_to=userId	HTTP 200, solo segnalazioni dell'operatore
R08	T05.3	Nessuna segnalazione	GET /api/reports/active per cittadino senza segnalazioni.	GET /api/reports/active (nessuna)	HTTP 200, array JSON vuoto []	HTTP 200, [] restituito
R08	T05.4	Accesso non autenticato	GET /api/reports/active senza cookie.	GET /api/reports/active (no auth)	HTTP 401, {"error": "unauthorized"}	HTTP 401 ricevuto
R08	T05.5	JSON senza Content-Type (BUG)	POST /api/report/respond inviando body JSON anziché application/x-www-form-urlencoded:	body={"report_id":5,"action":"accept"} senza header Content-Type corretto	HTTP 200, {"ok":true}	ERRORE : HTTP 400 missing params — report_id e action non

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
			get_field() non trovava i parametri perché il body non era URL- encoded.			venivano estratti dal body JSON
R0 8	T0 5.6	JSON con Content- Type corretto (FIX)	Usando il Content- Type corretto (application/ x-www-form- urlencoded) e body URL- encoded, la richiesta viene elaborata correttament e.	report_id=5&action=accept (URL-encoded)	HTTP 200, {"ok":true}	HTTP 200, accettazi one eseguita corretta mente
R0 9	T0 5.7	Segnala zioni archivia te	GET /api/reports/ archived con sessione valida.	GET /api/reports/archived + sessione	HTTP 200, JSON array status=3	HTTP 200, segnalazi oni risolte restituite
R19 — Accettazione o rifiuto dell'assegnazione (operatore)						
R1 9	T0 6.1	Accettaz ione valida	POST /api/report/re spond action=accept su segnalazion e ASSIGNED assegnata all'operatore .	report_id=5, action=accept	HTTP 200, {"ok":true}, status→2	HTTP 200, status aggiorna to a IN_PROG RESS
R1 9	T0 6.2	Rifiuto valido	POST /api/report/re spond action=reject :	report_id=5, action=reject	HTTP 200, {"ok":true}, status→0, assigned_to= NULL	HTTP 200, status 0 e assegnaz

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
			segnalazione e torna ACTIVE con assigned_to=NULL.			ione azzerata
R1 9	T0 6.3	Accept segnala zione altrui	POST /api/report/re spond su segnalazion e assegnata a un altro operatore.	report_id=7 (non assegnato a me)	HTTP 409, not assigned to you	HTTP 409 ricevuto
R1 9	T0 6.4	Action non valida	POST /api/report/re spond con action=delet e (non previsto).	action=delete	HTTP 400, action must be accept or reject	HTTP 400 ricevuto
R1 9	T0 6.5	Accesso da cittadino	POST /api/report/re spond con sessione cittadino.	sessione cittadino	HTTP 403, forbidden	HTTP 403 ricevuto
R11 — Risoluzione di una segnalazione (operatore)						
R1 1	T0 7.1	Risoluzi one valida	POST /api/report/st atus status=3 su segnalazion e IN_PROGR ESS dell'operator e.	report_id=5, status=3	HTTP 200, {"ok":true}, status→RES OLVED	HTTP 200, segnalazi one risolta
R1 1	T0 7.2	Risoluzi one non propria	POST /api/report/st atus su segnalazion e assegnata ad altro operatore.	report_id=8 (altrui)	HTTP 403, not authorized	HTTP 403 ricevuto
R1	T0	Risoluzi	POST	report_id=3 (status=3)	HTTP 403,	HTTP

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
1	7.3	one già risolta	/api/report/status su segnalazione già RESOLVED.		not in progress	403 ricevuto
R10, R12, R13 — Gestione segnalazioni e operatori (admin)						
R10	T08.1	Tutte le segnalazioni	GET /api/reports/all con sessione admin: tutte le segnalazioni del comune.	GET /api/reports/all + sessione admin	HTTP 200, JSON tutti gli stati	HTTP 200, array completo restituito
R10	T08.2	Accesso operatore	GET /api/reports/all con sessione operatore: negato.	GET /api/reports/all + operatore	HTTP 403, forbidden	HTTP 403 ricevuto
R13	T08.3	Lista operatori	GET /api/operators con sessione admin.	GET /api/operators + sessione admin	HTTP 200, JSON array operatori	HTTP 200, lista corretta per città
R12	T08.4	Assegnazione valida	POST /api/admin/assign con report ACTIVE e operatore valido della stessa città.	report_id=2, operator_id=9	HTTP 200, {"ok":true}, status→ASSIGNED	HTTP 200, assegnazione salvata
R12	T08.5	Operatore altra città	POST /api/admin/assign con operatore di città diversa.	operator_id=15 (altra città)	HTTP 400, invalid operator	HTTP 400 ricevuto
R12	T08.6	Report già in progress	POST /api/admin/assign su	report_id=5 (status=2)	HTTP 409, already in	HTTP 409

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
		s	segnalazione IN_PROGRESS.		progress	ricevuto
R1 2	T0 8.7	Report inesistente	POST /api/admin/assign con report_id inesistente.	report_id=99999	HTTP 404, not found	HTTP 404 ricevuto
R14, R18 — Caricamento e consultazione dati geografici						
R1 4	T0 9.1	Lista comuni	GET /api/cities (pubblico): restituisce l'array JSON dei comuni.	GET /api/cities	HTTP 200, JSON array di stringhe	HTTP 200, array comuni restituito
R1 8	T0 9.2	Validazione geo OK	POST /submit con coordinate interne al bounding box del comune.	lat=41.12, lon=16.87 (Bari)	HTTP 302, segnalazione accettata	HTTP 302, record inserito
R1 8	T0 9.3	Validazione geo KO	POST /submit con coordinate fuori dal bounding box.	lat=0.0, lon=0.0 (null island)	HTTP 200, errore coordinate	HTTP 200, errore bounding box visualizzato
R15, R16, R17 — Persistenza, sicurezza e gestione errori						
R1 5	T1 0.1	Persistenza riavvio	Dopo il riavvio del server, utenti e segnalazioni sono presenti.	Riavvio + GET /api/reports/all	HTTP 200, dati presenti	HTTP 200, tutti i dati persistiti
R1 6	T1 0.2	Rate limiting	IP che supera	Burst > 10000 req/s	HTTP 429, Too Many	HTTP 429

Co d. Re q.	Co d. Te st	Nome	Descrizione test	Input	Risultato atteso	Risultato ottenuto
			RATE_LIMIT_RPS richieste/s riceve 429.		Requests	ricevuto al superamento soglia
R1 7	T1 0.3	404 path inesistente	GET su path non registrato nella dispatch table.	GET /pagina_inesistente	HTTP 404 con il path nella risposta	HTTP 404 ricevuto
R1 7	T1 0.4	405 metodo errato	POST su rotta solo- GET (/api/reports/ active).	POST /api/reports/active	HTTP 405, Method Not Allowed	HTTP 405 ricevuto
R1 7	T1 0.5	403 ruolo sbagliato	GET /api/reports/ all con sessione operatore.	GET /api/reports/all + operatore	HTTP 403, forbidden	HTTP 403 ricevuto

Tabella 5. Risultati dei test funzionali.

6.2 Test di sicurezza

I test di sicurezza verificano la resistenza del sistema ai principali vettori di attacco web (XSS, SQL Injection) e la corretta gestione della memoria (memory leak rilevati con Valgrind). I test XSS e SQL Injection sono tutti superati grazie all'uso sistematico di prepared statement e della funzione `html_escape()`. I test di memoria documentano due bug reali nello shutdown del server, poi corretti.

C od e R eq.	Cod e Tes t	Nome	Descrizione e test	Input	Risultato atteso	Risultato ottenuto
XSS — Cross-Site Scripting						
S E C	TS0 1.1	XSS in username (registrazione)	Registrazione con username contenente tag script: il	username=<script>alert(1)</script>	Username visualizzato come testo: <script>alert(1)<	HTML escaped correttamente e da html_escape()

Codice Req.	Codice Test	Nome	Descrizione e test	Input	Risultato atteso	Risultato ottenuto
			sistema deve effettuare l'escape prima di iniettarlo nel template HTML.		;/script>	)
SEC	TS0 1.2	XSS in description (segnalazione)	Segnalazione con description contenente tag HTML: deve essere conservata nel DB inalterata ma visualizzata escaped nel JSON/HTML.	description=	Stringa salvata nel DB, escaped nell'output HTML	Output: ;, nessuno script eseguito
SEC	TS0 1.3	XSS in category	Segnalazione con category contenente payload XSS.	category=<svg onload=fetch('//evil')>	Escape corretto, nessuna esecuzione del payload	Valore escaped nel template, nessuna richiesta a //evil
SEC	TS0 1.4	XSS nel messaggio di errore	Tentativo di iniettare script tramite parametro URL che finisce nel messaggio di errore 404.	GET /pagina<script>alert(1)</script>	HTTP 404, path escaped nell'HTML della risposta	Path reso safe da html_escape() prima di sprintf
SQL Injection						

Cod. Req.	Cod. Test	Nome	Descrizione e test	Input	Risultato atteso	Risultato ottenuto
SEC	TS0 2.1	SQLi in username (login)	Tentativo di bypass autenticazione con SQL injection classica nel campo username.	username=' OR '1'='1	HTTP 200, «Username o password errati» — nessun bypass	Prepared statement: il valore è bind come parametro, non concatenato
SEC	TS0 2.2	SQLi in password (login)	Tentativo di commentare la clausola WHERE con -- nel campo password.	password=abc' --	HTTP 200, errore autenticazione — nessun bypass	SHA256 applicato prima della query: il hash non coincide mai
SEC	TS0 2.3	SQLi in report_id	Tentativo di iniezione nel campo report_id dell'API /api/report/status.	report_id=1 OR 1=1	HTTP 400 o 404 — strtoull() converte a 1, guardia WHERE non soddisfatta	strtoull() ignora la parte ' OR 1=1'; binding con 'l' int64, nessuna iniezione
SEC	TS0 2.4	SQLi in city (registrazione)	Tentativo di iniezione nel campo city durante la registrazione.	city=Bari'; DROP TABLE users; --	HTTP 200, «Comune non riconosciuto» — nessuna query eseguita su quel valore	geo_lookup() fallisce prima che il valore raggiunga il DB; prepared statement
SEC	TS0 2.5	SQLi in description (segnalazione)	Tentativo di iniezione nella descrizione di una segnalazione.	description=Buca'; DELETE FROM reports; --	HTTP 302, segnalazione inserita con descrizione letterale	db_exec() con binding 's': valore passato come stringa, nessuna iniezione
Memory leak — Valgrind (shutdown)						
S	TS0	Leak	Test con	Valgrind --leak-	Valgrind report: 0 byte	FALLITO:

Codice Req.	Codice Test	Nome	Descrizione e test	Input	Risultato atteso	Risultato ottenuto
EC	3.1	risorse	Valgrind sul processo server: allo shutdown, i prepared statement SQLite non venivano finalizzati prima di sqlite3_close(), lasciando 3 handle aperti.	check=full sul server con shutdown via SIGINT	definitely lost	risultano blocchi still reachable da sqlite3_close() non conclusivo e allocazioni eseguite per hash table
SEC	TS0 3.3	Leak risorse (dopo correzione e shutdown)	Dopo la correzione dello shutdown e della gestione dei segnali (SIGINT/SIGTERM)	Valgrind --leak-check=full sul server	Valgrind: 0 bytes in 0 blocks for definitely lost e indirectly lost	SUPERATO: Valgrind riporta 0 definitely lost, 0 indirectly lost. Tutto ciò che è stato allocato durante la vita del programma è stato liberato correttamente

Tabella 6. Risultati dei test di sicurezza e memoria.

6.3 Test di performance

6.3.1 Metodologia

I test di performance sono stati eseguiti con lo strumento wrk, un moderno benchmark HTTP multi-threaded. Ogni iterazione è stata condotta con 2 thread, 100 connessioni simultanee e una durata di 10 secondi, ruotando tra 4 endpoint (GET /api/reports/active, GET /api/reports/archived, GET /api/cities, GET /home) tramite lo script Lua descritto di seguito. Le ottimizzazioni sono state applicate in sequenza: ogni iterazione incorpora le precedenti.

6.3.2 Script wrk utilizzato

```
-- Ruota tra 4 endpoint ad ogni richiesta
```

```

local endpoints = {
    '/api/reports/active',
    '/api/reports/archived',
    '/api/cities', '/home',
}

local idx = 0

request = function()
    idx = (idx % #endpoints) + 1

    wrk.headers['Cookie'] = 'session=<token_valido>'

    return wrk.format('GET', endpoints[idx])
end

-- Comando utilizzato per ogni iterazione:
-- wrk -t2 -c100 -d10s -s script.lua http://localhost:8080
-- -t2: 2 thread -c100: 100 connessioni simultanee -d10s: 10 secondi

```

Listato 16. Script Lua per wrk — benchmark su 4 endpoint con 100 client simultanei.

6.3.3 Storico risultati

La tabella seguente riporta lo storico delle misurazioni. Le righe in rosso indicano performance insoddisfacenti (< 5 000 req/s), le righe in verde indicano risultati accettabili e molto buoni.

Iter.	Ottimizzazione applicata	Endpoint testato	Req/s	Lat. media	Lat. max
v1	Baseline — nessuna ottimizzazione	GET /api/reports/active	~1 100 req/s	46 ms	76 ms
v2	Parser HTTP zero-copy (picohttpparser) + serializzazione cJSON	GET /api/reports/active	~4 200 req/s	23 ms	42 ms
v3	Cache JSON dei report (report_cache)	GET /api/reports/active	~8 600 req/s	11 ms	43 ms

Iter.	Ottimizzazione applicata	Endpoint testato	Req/s	Lat. media	Lat. max
v4	Buffer di risposta statici e preallocati, + istruzioni compilatore (allineamento, macro branch prediction)	GET /api/reports/active	~17 000 req/s	2.7 ms	29 ms

Tabella 7. Storico dei test di performance con wrk (4 thread, 100 connessioni, 30 s).

Schermata 1. Output wrk — v1 — Baseline (~1 100 req/s)

```

wrk -t2 -c100 -d10s -s analisi/bench.lua http://localhost:8080
Running 10s test @ http://localhost:8080
 2 threads and 100 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
| Latency    46.45ms   4.30ms   76.57ms   88.16%
| Req/Sec    1.08k     82.84    1.22k     74.50%
11430 requests in 10.02s, 24.87MB read
Requests/sec: 1139.48
Transfer/sec: 2.48MB

```

Schermata 2. Output wrk — v2 —picohttpparser + cJSON (~4 200 req/s)

```

$ wrk -t2 -c100 -d10s -s script.lua http://localhost:8080
Running 10s test @ http://localhost:8080
 2 threads and 100 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
| Latency    23.31ms   2.66ms   42.02ms   88.68%
| Req/Sec    2.14k    200.45    2.53k     71.50%
42698 requests in 10.03s, 16.82MB read
Requests/sec: 4258.06
Transfer/sec: 1.68MB

```

Schermata 3. Output wrk — v3 — Cache JSON report (~8 600 req/s)

```

└─$ wrk -t2 -c100 -d10s -s script.lua http://localhost:8080
Running 10s test @ http://localhost:8080
 2 threads and 100 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency    11.70ms   5.22ms  43.83ms   96.54%
  Req/Sec    4.38k    663.49   5.05k    92.00%
87212 requests in 10.04s, 142.14MB read
Requests/sec: 8685.89
Transfer/sec: 14.16MB

```

Schermata 4. Output wrk — v4 — Buffer statici preallocati (~17 000 req/s)

```

└─$ wrk -t2 -c100 -d10s -s script.lua http://localhost:8080
Running 10s test @ http://localhost:8080
 2 threads and 100 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency     2.72ms   3.06ms  29.49ms   94.88%
  Req/Sec    9.02k    628.53   9.52k    85.50%
180090 requests in 10.06s, 105.20MB read
Requests/sec: 17897.13
Transfer/sec: 10.45MB

```

6.4 Test di copertura — Icov

Presente in [allegato](#).

- Strumento: gcov (GCC) + Icov 2.x + genhtml per il report HTML.
- Flag di compilazione: -fprofile-arcs -ftest-coverage (-O0 per evitare ottimizzazioni che eliminano branch).
- Comando raccolta: Icov --capture --directory . --output-file coverage.info
- Comando report: genhtml coverage.info --output-directory coverage_report/

Conclusioni

Il testing è stato diviso in 4 aree: requisiti funzionali, sicurezza, performance e copertura codice. I 55 test funzionali hanno verificato ogni caso d'uso definito nell'analisi, con attenzione particolare agli scenari di errore e ai vincoli di ruolo. Su 55 test, 51 hanno prodotto il risultato atteso al primo tentativo. Solo 4 test che hanno rivelato comportamenti errati hanno documentato bug reali poi corretti.

I test su XSS e SQL Injection hanno confermato la solidità delle difese implementate. L'uso sistematico di prepared statement SQLite con binding parametrizzato rende impossibile qualsiasi forma di SQL injection indipendentemente dal valore dei campi in ingresso. L'escaping applicato a tutti i valori utente prima dell'iniezione nei template HTML impedisce l'esecuzione di script malevoli nel browser. Il confronto a tempo costante (constant_time_compare) per la verifica delle password previene i timing attack.

I test di memoria con Valgrind hanno rivelato bug (memory leak) nello shutdown: sono stati corretti implementando uno shutdown ordinato ponendo attenzione alla gestione dei segnali.

Il percorso di ottimizzazione ha prodotto un miglioramento complessivo di $\sim 8\times$ tra versione iniziale e finale. Ogni raffinamento ha introdotto un'ottimizzazione mirata portando a misure ottime per un server single-threaded.

L'analisi lcov/gcov ha evidenziato una ottima copertura del codice e delle funzioni in particolare. I numeri meno soddisfacenti sono causati dai path di errore che difficilmente possono avvenire oppure da funzioni che sono inutilizzate, ma che possono essere utili nel futuro (hash table).

Tra i punti di forza si riconoscono l'utilizzo di tecniche avanzate e moderne del C e , in generale, della programmazione a basso livello: lo slab allocator personalizzato con header pointer elimina la frammentazione heap e garantisce alloc/free in $O(1)$ puro per le strutture ClientCtx, evitando il costo del memory allocator di sistema sotto carico; la struttura dati custom dell'hash table thread-safe (caratteristica utile per sviluppi futuri); attenzione alla sicurezza con meccanismi di salting (password) e seeding (hash table).

La tabella seguente riassume i punti di forza, le debolezze e gli sviluppi futuri pianificati.

✓ Punti di forza	✗ Debolezze	→ Sviluppi futuri
Tecniche avanzate di gestione memoria: slab allocator personalizzato con header pointer nascosto, alloc/free $O(1)$ senza frammentazione heap	Single-thread: query SQLite lente o picchi di I/O bloccano il loop libuv per tutte le connessioni attive	Multithreading: worker thread pool con uv_queue_work per isolare le query SQLite dal loop libuv, eliminando il blocco single-thread
Strutture dati avanzate: hash table thread-safe e liste	Rate limiting a sliding window con soli due contatori:	Rate limiting avanzato con LRU
Architettura event-driven single-thread con libuv:, gestione di migliaia di connessioni con un solo thread	Cache report con chiave quadrupla e TTL fisso: non considera la frequenza di accesso né la priorità per città	Cache report sofisticata: LRU esatta con lista concatenata separata
Performance ottimali per un server single-threaded: $\sim 17\,000$ req/s con latenza media < 3 ms su 100 connessioni simultanee	Portabilità limitata: dipendenza da POSIX (pthread, /dev/urandom, mmap), non compilabile su Windows senza adattamenti	Portabilità migliorata: astrazione del layer OS (CMake cross-platform, sostituzione /dev/urandom con API crittografiche standard)
Cache JSON in-memory con TTL, invalidazione per città e politica LRU-approssimata in $O(n)$	Bounding box geografico approssimato (AABB): può accettare coordinate esterne al perimetro reale del comune	Uso di API apposite per eliminare i falsi positivi del bounding box
Sicurezza: prepared statement SQLite per SQL injection prevention, html_escape() per	Assenza di HTTPS nativo	HTTPS nativo

XSS, seeding per hash table, salting per le password, constant_time_compare() contro timing attack		
Dati ausiliari per-richiesta gestiti in-memory con hash table $O(1)$: sessioni, IP per rate limiting e coordinate geografiche		Notifiche in tempo reale agli operatori per nuove assegnazioni
		Validazione della registrazione basata sui vari tipi di utenti e delle segnalazioni